Degree Project in the Field of Technology and the Main Field of Study Computer Science and Engineering

Second cycle, 30 credits

Automated Generation of Visual Regression Tests Through State Exploration

CHARLOTTA JOHNSSON

Automated Generation of Visual Regression Tests Through State Exploration

CHARLOTTA JOHNSSON

Master's Programme, Computer Science, 120 credits
Date: June 18, 2025

Supervisors: Deepika Tiwari, William Perkola
Examiner: Martin Monperrus
School of Electrical Engineering and Computer Science
Host company: OSTTRA
Swedish title: Automatiserad generering av visuella regressionstester genom tillståndsutforskning

Abstract

As web applications become increasingly central to digital experiences, ensuring visual consistency across user interfaces is crucial. Unintended visual changes, or visual regressions, can harm user experience and lead to significant issues for organisations. Visual Regression Testing (VRT) helps address this by detecting visual anomalies during the development process. This study explores how state exploration can enhance VRT suites for React component libraries using Storybook and Playwright. We present AutoVRT, a novel tool that automatically generates interaction-based VRT tests and evaluates them using first-order mutation analysis. Applied to five components from an industrial project: OSTTRA's React library, AutoVRT produced 86 new interaction-based tests, improving the mutation score from 0.22 to 0.53, a 130% increase per component on average. Remaining limitations suggest a need for more well-defined component stories to support more effective test generation.

Keywords

Visual Regression Testing, Mutation Testing, Software Testing, State Exploration

Sammanfattning

I en tid där webbapplikationer spelar en central roll i digitala tjänster blir visuell enhetlighet allt viktigare. Oavsiktliga visuella förändringar, så kallade visuella regressioner, kan resultera i en sämre användarupplevelse, vilket i sin tur kan få en negativ effekt för organisationer. Visuell regressionstestning (VRT) är en etablerad metod för att upptäcka sådana avvikelser under utvecklingsprocessen. Den här studien undersöker hur tillståndsutforskning kan förbättra effektiviteten hos VRT-sviter för React-komponentbibliotek, genom att använda Storybook och Playwright. Vi presenterar det utvecklade verktyget AutoVRT, som automatiskt genererar interaktionsbaserade VRT-tester och utvärderar deras förmåga att upptäcka fel med hjälp av mutationstestning. AutoVRT tillämpades på fem komponenter från det industriella projektet OSTTRA:s React-bibliotek, vilket resulterade i 86 nya tester. Andelen upptäckta mutationer förbättrades från 0,22 till 0,53, vilket motsvarar en genomsnittlig ökning på 130% per komponent. Trots förbättringarna pekar resultaten på ett fortsatt behov av tydligare och mer heltäckande komponentversioner (stories) för att möjliggöra effektiv testgenerering och tillståndsutforskning.

Nyckelord

Visuell Regressionstestning, Mutationstestning, Mjukvarutestning, Tillstånds-utforskning

Acknowledgments

I would like to express my gratitude to my closest friends and family for their unwavering support throughout my degree. A special thanks to my partner, Kristoffer Gunnarsson, who has been my emotional anchor during these challenging times. I would also like to thank my supervisor at OSTTRA, William Perkola, for his steadfast support and positive outlook, as well as for his valuable advice. Additionally, I am grateful to my KTH supervisor, Deepika Tiwari, for reviewing countless drafts and providing constructive feedback. Lastly, I would like to extend my thanks to my examiner, Martin Monperrus.

Stockholm, June 2025
Charlotta Johnsson

Contents

1 Introduction **1**
1.1 Problem Statement . . . 2
1.2 Thesis Context . . . 3
1.3 Research Methodology . . . 4
1.4 Thesis Contribution . . . 5
1.5 Structure of the Thesis . . . 5

2 Background **7**
2.1 Visual Regression Testing . . . 7
2.2 React Component Library . . . 9
2.3 Storybook . . . 10
2.4 Playwright . . . 12
2.4.1 Visual Tests . . . 13
2.4.2 Interaction in Tests . . . 14
2.5 Mutation Testing . . . 15
2.6 Related Work . . . 16
2.6.1 Visual Regression Testing . . . 17
2.6.2 Mutation Testing . . . 18
2.6.3 Novelty Statement . . . 19

3 Design & Implementation **21**
3.1 Overview of AutoVRT . . . 21
3.1.1 Phase I: Test Generation . . . 22
3.1.1.1 Stories Unit . . . 22
3.1.1.2 Action Unit . . . 23
3.1.1.3 Screenshot Unit . . . 24
3.1.2 Phase II: Mutation Analysis . . . 28
3.1.3 End-to-End Example . . . 29
3.1.3.1 Phase I . . . 30

3.1.3.2 Phase II 33

4 Experimental Methodology **35**
4.1 Research Questions 35
4.2 Study Subject 36
4.2.1 Experimental Setup 39
4.2.1.1 Original VRT suite 39
4.3 Protocol 39
4.3.1 RQ1: Mutation Operators for VRT suites 39
4.3.2 RQ2: Mutation Score of the Original VRT suite 40
4.3.3 RQ3: Selecting Actions for Test Generation 40
4.3.4 RQ4: Comparative Analysis Between the Original and Improved VRT Suite 40

5 Experimental Results and Analysis **43**
5.1 RQ1: Mutation Operators for VRT suites 43
5.2 RQ2: Mutation Score of the Original VRT suite 46
5.3 RQ3: Selecting Actions for Test Generation 49
5.4 RQ4: Comparative Analysis Between the Original and Improved VRT Suite 52

6 Discussion **59**
6.1 State Exploration for Test Generation 59
6.2 Generalisability of Mutation Operators 61
6.3 Fault Detection Abilities of Visual Regression Test Suites . . . 62

7 Conclusions and Future Work **67**
7.1 Conclusions 67
7.2 Limitations 68
7.3 Future Work 68

References **71**

List of Figures

2.1 Comparison of CSS before and after a visual regression 8
2.2 The effects of a visual regression impacting the bottom-margin of a Switch component. 8
2.3 Example of React components being combined to create a form. 11
2.4 Example of a viewing a component in Storybook 12
2.5 Comparison of the function isValidUser before and after introducing a mutation 16

3.1 A structural overview of AutoVRT. 22
3.2 Flow chart of the Stories unit 23
3.3 Flow chart of Action unit 25
3.4 Flow chart of the Screenshot loop 26
3.5 The initial state of the Switch component 30
3.6 The state of the Switch after clicking the label 31
3.7 The state of the Switch after clicking the label and then the switch 32

4.1 Displaying examples of one state components from OSTTRA's component library. 37
4.2 Depicting the initial state and the state after interaction for an Accordion component from OSTTRA's Component library. . . 37

5.1 Showing a mutant that alters the colour of the Dropdown caret. 47
5.2 Examples of single-state components from the OSTTRA React component library. 49
5.3 Demonstrates how a visual indicator signals when the focus is on the textInput component. 49
5.4 Mutation scores for the VRT before and after the addition of generated tests. 54
5.5 Venn diagram of detected mutants 54

5.6 Code and visual states of the Accordion component. 55
5.7 Example of mutants not detectable by the VRTs. 56

List of Tables

2.1 Supported Locators in the Locators API 14
2.2 The most commonly used Playwright actions 15

4.1 OSTTRA React component library: Static vs Interactive . . . 38

5.1 The 10 most common CSS properties and their frequency in the OSTTRA React component library. Highlighted rows represent colour-related properties. 44
5.2 The number of stories, mutants and the corresponding mutation scores for each component of the original VRT. . . . 47
5.3 Interaction support among components in the OSTTRA library, including percentage relative to the total number of components. 51
5.4 Supported interactions per component 51
5.5 Number of mutants and generated tests, along with mutation scores, for each component before and after integrating the generated tests into the VRT suite. 53

Listings

2.1 The base structure of a visual test using Playwright. 13
2.2 Example of an interaction using Playwright. 14
3.1 The code used for test generation 25
3.2 Original CSS code of the Avatar component before the first mutation script 28
3.3 Mutated version 1 of the Avatar CSS 28
3.4 Mutated version 2 of the Avatar CSS 29
3.5 The baseline test for switch standard 30
3.6 The generated test that clicks the label of the Switch component 31
3.7 The generated test that clicks the label and the switch of the Switch component 32
3.8 The CSS of the Switch compoennt 33
5.1 The CSS of the Avatar component. The occurrences of colour-related properties are highlighted in grey 45
5.2 The CSS of the DropDown component. Relevant lines are highlighted in grey 47
5.3 Original CSS 55
5.4 CSS where the color property has been mutated 55
5.5 Generated test that killed the mutation 55

Chapter 1

Introduction

In recent years, the speed of software development has accelerated due to agile methodologies, continuous integration, and frequent development cycles. Web pages are developed and updated more frequently, increasing the risk of introducing unintended defects into the code. In this environment, the need for visual consistency and integrity has become more critical [1].

Evolving accessibility standards and regulations drive the demand for well-functioning user interfaces (UIs). New regulations, such as the European Accessibility Act (EAA), which comes into effect on June 28th of this year, mandate that digital products in several fields be inclusive and visually accessible for all users, including those with disabilities [2]. Legislation like this increases the pressure on developers to ensure that the UI remains both consistent and accessible throughout the development cycle.

To address the challenges associated with visual defects, Visual Regression Testing (VRT) has gained significant traction in recent years. VRT is designed to identify unintended changes in the visual presentation of user interfaces, thereby ensuring a reliable and consistent user experience. One approach involves comparing screenshots of pages or individual elements across different application versions, enabling the detection of variations such as element shifts and colour changes [3, 4].

However, as components evolve to respond more dynamically to user interactions, their visual behaviour can change, adding a layer of complexity to the detection of visual discrepancies. This variability may hinder the

identification of issues that could adversely affect the user experience.

Furthermore, contemporary web pages comprise a wide range of components that create intricate user interfaces. This complexity presents notable testing challenges, as individual components or combinations can lead the page to enter multiple states, both visually and functionally [5]. Each state may offer distinct presentation and interaction opportunities, underscoring the necessity for comprehensive testing to ensure consistency across all scenarios.

1.1 Problem Statement

In this section, we present the problem statement, outline the research questions, and address the scientific and engineering challenges associated with the study.

Currently, there are numerous open-source tools available for performing VRT. For example, Playwright [6], Chromatic [7], and Percy [8] are widely used to detect visual changes. While tools like this have advanced significantly over the last years, they often struggle to effectively manage the visual changes that arise from user interactions. Some tools support manually defining the possible actions for each test [9]. However, this approach is both time-intensive and challenging, as it may be difficult to account for all possible combinations of actions. The inability to cover all actions may lead to false negatives within the regression test suite, which can compromise the reliability of the tests. [9].

A notable challenge with manually defining actions is that many sequences can produce identical visual outcomes. Developers often find it difficult to predict which actions will result in distinct visual states, making the creation of individual tests for every action or combination that creates distinct states challenging [9].

In addition, creating a test for every possible permutation of action sequences would lead to an overwhelming number of tests that essentially evaluate the same outcome, resulting in impractical running times and making the test suite hard to manage. Large regression test suites can hinder adequate regression testing, especially when there is a strict delivery schedule [10]. This can adversely affect both the quality and reliability of the software, highlighting the importance of minimising the number of tests to avoid excessive testing.

Consequently, there is a pressing need for more efficient processes or tools that can identify and manage these visual states through automated means, reducing the reliance on manual intervention and enhancing test coverage. Otherwise, the regression suites may be abandoned if they become too difficult to maintain, since the VRT suite needs to be manually updated along with any changes to the source code to ensure the VRT is up to date.

Building on closely related work [9], this thesis investigates the feasibility of automatically generating tests by identifying distinct states through state exploration and visual comparison, thereby enhancing a VRT suite. By identifying actions that produce significant visual differences, these test variations can be incorporated into the regression test suite, ultimately increasing test coverage and improving the suite's effectiveness in detecting visual regressions.

Problem Statement

Modern VRT tools like Playwright, Chromatic, and Percy struggle with handling user interactions. Manually defining action sequences is time-consuming, prone to omissions, and often results in redundant tests due to visually equivalent states. As a result, there is a critical need for automated methods that can efficiently explore visual states and avoid generating excessively large test suites, ensuring VRT remains practical and scalable. The thesis investigates these topics.

1.2 Thesis Context

This thesis is conducted at OSTTRA AB, a financial technology company specialising in post-trade services for the global financial markets. OSTTRA provides a range of products, including multilateral portfolio compression, risk optimisation, and trade reconciliation. Their solutions are designed to reduce costs, improve capital efficiency, and mitigate systemic risk in post-trade workflows.

For OSTTRA, maintaining visual consistency and reliability across their various products is of significant importance. To achieve this, they, among other things, utilise a React component library to ensure that their products have a uniform appearance and behaviour. It is crucial for them that this

component library is thoroughly tested to guarantee high-quality services. As such, it serves as an appropriate subject for study in this thesis.

1.3 Research Methodology

In order to address the problem statement, we develop a tool called AutoVRT (Auto Visual Regression Testing), which automatically generates visual regression tests. To assess the ability of AutoVRT to generate impactful tests, we evaluate it against a real-world project, OSTTRA's React component library. For an effective evaluation of AutoVRT and the tests it generates, we must first identify suitable mutants for the UI components in the component library in RQ1. In RQ2, we will examine the fault-detecting capabilities of the existing tests for the component library. This will serve as a baseline to determine whether the tests generated by AutoVRT actually improve the test suite. The purpose of RQ3 is to explore which user actions should be utilised by AutoVRT when it interacts with the OSTTRA React component library to generate tests. Finally, in RQ4, we will compare the effectiveness of the generated tests at detecting the introduced mutants with that of the baseline established in RQ2.

- RQ1: What types of mutants are most relevant to VRT in terms of their prevalence and effectiveness in evaluating VRT suite quality?
- RQ2: What is the current mutation score of the original VRT suite?
- RQ3: What are the most common UI actions for the React Component Library?
- RQ4: How effectively can automated visual regression test generation using state exploration enhance the mutation score of a VRT suite?

The research methodology we use to address the research questions is organised into three parts.

1. **Component Library Analysis:** We analyse the source code of the component library and interact with its components to understand their behaviour and structure.

2. **AutoVRT Implementation:** Using insights from the component analysis, we develop a tool that automatically generates and evaluates VRTs containing interactive elements.

3. **Execution and Evaluation:** We run AutoVRT to generate and assess the effectiveness of the VRTs.

1.4 Thesis Contribution

The contributions of this work are as follows:

- We develop AutoVRT, a novel tool for automated visual regression testing of UI component libraries.
- We design a two-phase process for AutoVRT.
  - Phase I: Systematically explores and interacts with UI components to generate tests that cover untested visual states.
  - Phase II: Evaluates the generated tests based on their fault-detection capabilities, by introducing controlled visual faults.
- We demonstrate how AutoVRT can be used to create new or enhance existing VRT suites, improving test coverage and effectiveness for UI component libraries.

1.5 Structure of the Thesis

The structure of the thesis is as follows:

In Chapter 2, we provide background information on Visual Regression Testing, Mutation Testing, React component libraries, and the tools utilised throughout the thesis. Additionally, we include a summary of previous work in the field.

In Chapter 3, we present AutoVRT, the technical contributions of the thesis. We explain the practical details of the implementation of AutoVRT.

In Chapter 4, we discuss the motivations behind the research questions, the subjects of our study, and the protocol employed to address these research questions.

Chapter 5 presents the results of the thesis, including the mutation score of the Visual Regression Testing suites and other significant findings.

Finally, Chapter 6 contains a discussion of the results, ending in the conclusions and suggestions for future work in this area, detailed in Chapter 7.

Chapter 2

Background

This chapter presents the theoretical background of the thesis. We start by outlining the relevant methods and tools, followed by a discussion of previous work in the research area. Finally, we conclude with a summary of the chapter.

2.1 Visual Regression Testing

A visual regression is an unintended change that results in a modification in the appearance of the UI [3]. Visual Regression Testing (VRT) is a software testing technique used to detect visual regressions. One technique that is used to detect visual regressions is image comparison. Commonly, this is done by comparing screenshots before and after a code change to detect visual discrepancies [3, 11]. VRT ensures that new releases do not introduce visual inconsistencies, such as colour changes or element shifts. An example of this can be seen in Figure 2.1, where the CSS for a Switch component has undergone a visual regression, changing the bottom-margin property from 0 to 10 pixels (see Lines 4 and 4).

The impact of this regression is illustrated in Figure 2.2. In this figure, the actual appearance of the Switch with the regression (see Fig. 2.2b) is positioned slightly higher in the image compared to the expected appearance (see Fig. 2.2a). This difference becomes noticeable when the two images are viewed side by side, and is highlighted in orange in Figure 2.2c. However,

(a) Original CSS

```
label {
  display: inline-block;
  height: $switch-height;
  margin-bottom: 0;
  margin-right: $label-margin;
  line-height: $switch-height;
  ...
}
```

(b) CSS with visual regression

```
label {
  display: inline-block;
  height: $switch-height;
  margin-bottom: 10px;
  margin-right: $label-margin;
  line-height: $switch-height;
  ...
}
```

Figure 2.1: Comparison of CSS before and after a visual regression

this visual regression can be challenging to detect without such side-by-side comparison. This is one of the reasons why specialised VRT tools are utilised to help identify such unintended changes.

(a) Expected appearance (b) Actual appearance (c) Difference

Figure 2.2: The effects of a visual regression impacting the bottom-margin of a Switch component.

VRT tools, such as Playwright [6], Chromatic [7], Percy [8], BackstopJS [12] and Loki [13], share the common goal of detecting visual changes. Each tool approaches this challenge differently. However, one key issue that they all face is that the visual changes they aim to detect can be extremely subtle, often involving only a few pixels. Additionally, various factors such as differences in browsers, screen resolutions, and rendering engines can significantly impact an application's appearance, leading to unintended variations causing false

positives [14, 4].

Various techniques have been developed to balance precision and reliability to ensure the tool is sensitive enough to detect meaningful visual differences while minimising false positives. Examples of this include setting a tolerance for how many pixels can differ between two images, either by ratio or by number, allowing images with very small differences to pass while still being sensitive enough to catch important differences [6]. Another example involves perceptual matrices, which weigh the impact of pixel differences based on how noticeable the changes are to the human eye. Humans are more sensitive to certain colours; for instance, a pixel difference that includes a red pixel will be more noticeable than one that involves, say, a green pixel. The tool can then rank the severity of the differences according to the perceptual matrices to catch important variations [15].

Another common issue in VRT is dynamic content, which poses a challenge for image comparison methods and techniques. This can be, for example, animations or timestamps. This poses a challenge because images captured at different points in time can cause visual variations, which can be mistaken for a visual regression. To address this issue, a solution is to utilise masking techniques to ensure that the area does not interfere with the testing process [11]. Other alternatives to mitigate this problem include disabling animations or setting the time to a fixed value.

Finally, user interactions play a crucial role in influencing the visual appearance of the UI. Some tools, like Chromatic and Playwright, allow programmers to manually specify the types of interactions to be performed in each test. In contrast, other tools, such as Percy, support interactions only when used in conjunction with other testing frameworks like Selenium [16] or Cypress [17].

2.2 React Component Library

A React Component Library is a collection of frontend components written in React. There are many popular React component libraries, such as Material UI [18], Chakra UI [19], and Ant [20], to mention a few. The primary purpose of a component library is to gather commonly used components so that they can be reused. These libraries are essential for software companies as they help

maintain and streamline development, eliminating the need to rewrite code. This approach also simplifies testing and ensures a consistent appearance and behaviour across the product or even multiple products. By collecting components in a single location, it becomes easier to find different components and to build pages or larger components using the library's components as building blocks.

Figure 2.3 is a visual representation of how various components can be combined to build a form. The layout includes a Heading component (in green) that serves as a title or introduction to the form. Under the heading, there are two Select components (in yellow) that allow users to choose from a range of predefined options. Below these, there are two TextInput components (in blue) for users to enter their data. Next to them is a DatePicker component (in red) that enables users to select dates. Additionally, the form features a Checkbox group (in purple), which offers multiple options for users to select. Lastly, a Switch component (in dark green) provides a simple toggle mechanism for binary choices.

2.3 Storybook

Storybook is an open-source tool designed to serve as a workshop for developing user interface components and pages in isolation from the product [21]. It is frequently utilised for showcasing and interacting with component libraries. As organisations handle multiple products and web pages, the use of component libraries for front-end development has become more prevalent [22]. These libraries facilitate code reuse across various projects while maintaining consistency in functionality and design. Storybook enables users to view and interact with these components independently, and it also acts as documentation for the components, outlining their usage and different versions.

For each component, it is possible to manually define multiple versions, called "stories." Storybook organises components by creating a dedicated folder for each one, containing its various stories along with documentation. When viewing a specific story, Storybook allows interaction with the component and provides access to the underlying HTML elements. Users can inspect the markup and modify input parameters in real-time. An example of viewing

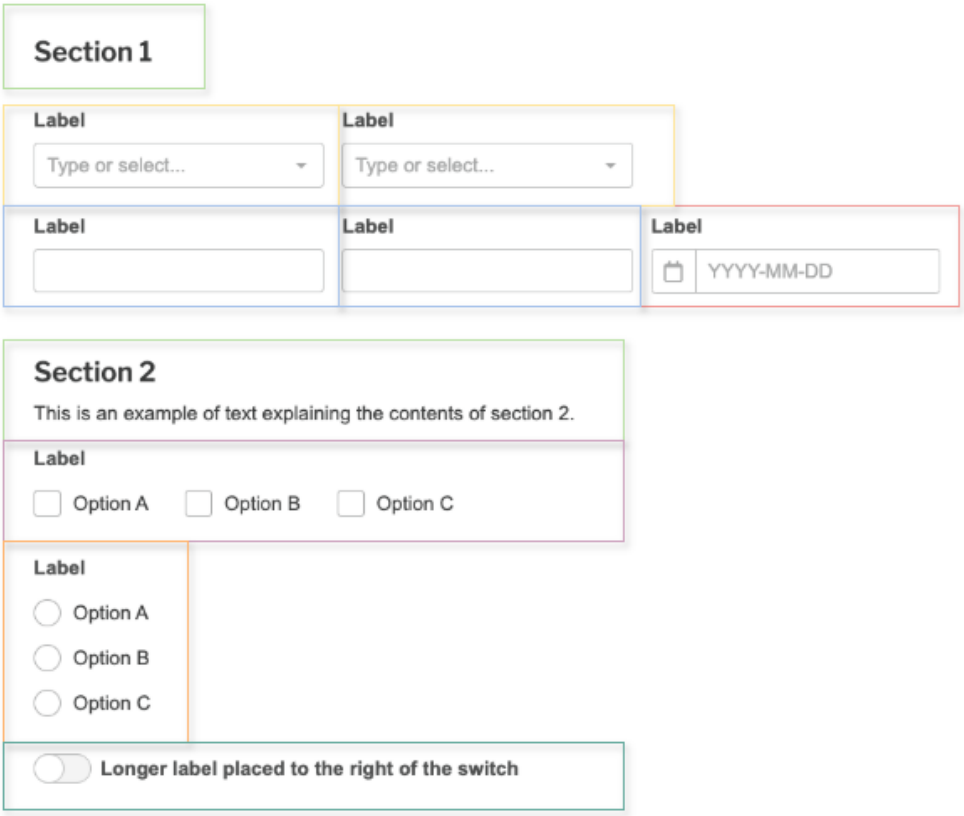


Figure 2.3: Example of React components being combined to create a form.

a component using OSTTRA's React component library in Storybook is illustrated in Figure 2.4.

In the figure, the left panel displays the available component folders, such as the Dropdown component shown here. Inside each folder, different versions can be found, such as Standard, Condensed, Staff and Disabled, along with the corresponding documentation. On the top right, the selected component (Dropdown) is rendered on the canvas, providing a live preview of how it looks and behaves. The control panel on the bottom right lets users inspect the HTML and interactively adjust input parameters, such as type or label, to test different inputs in real time. In this example, we can see that the Dropdown contains various HTML elements, including multiple divs and a button.

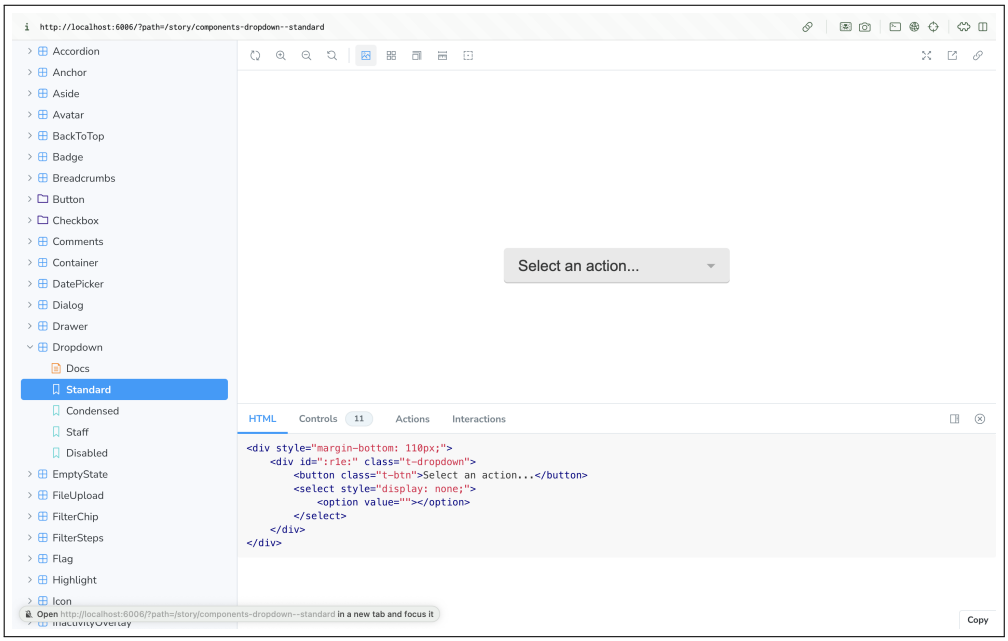


Figure 2.4: Example of a viewing a component in Storybook

2.4 Playwright

Playwright [6] is an open-source tool for web application testing created by Microsoft. It supports multiple web browsers, including Chromium, Firefox, and WebKit, which are among the most common browser engines [23]. Playwright facilitates both end-to-end testing as well as visual tests. To use

the Playwright framework, one configuration file and one or more test files are required. The configuration file enables the customisation of tests by specifying the folders to use, the types of browsers to run, the number of retries, the timeout, and other relevant settings.

2.4.1 Visual Tests

Playwright's visual tests, in their simplest form, are structured as follows (see Listing 2.1): The test first navigates to the URL of the web page that is being tested using its URL (Line 4). The tool then performs the actual visual comparison using Line 5. On first execution, this will generate a screenshot for each test. These screenshots are then saved in a folder and kept to use as a reference during subsequent runs. These images, referred to as "golden copies," serve as the baseline for how we expect our system to appear and remain unchanged.

Listing 2.1: The base structure of a visual test using Playwright.

```
1 import { test, expect } from '@playwright/test';
2
3 test('example test', async ({ page }) => {
4   await page.goto('https://playwright.dev');
5   await expect(page).toHaveScreenshot();
6 });
```

During subsequent runs, when Playwright detects that there are reference shots saved, the statement will generate a new screenshot. The new image will then be compared to the stored reference using the Playwright comparator. Playwright's comparator uses the Pixelmatch library, which is a JavaScript library that performs pixel-by-pixel comparisons [15]. Pixel-by-pixel comparison is a common technique in VRT where screenshots are analysed at the pixel level to determine if they are identical. While this method offers high precision, it is also susceptible to false positives, especially when minor rendering variations occur due to factors such as font smoothing or anti-aliasing. To mitigate these issues, features such as anti-aliasing effects are used. Anti-aliasing is a graphics technique that smooths the appearance of edges by blending pixel colours along boundaries. While visually beneficial, it can result in pixel-level differences that are not perceptible to users but can trigger test failures in strict comparison modes. Therefore, enabling anti-alias detection or setting pixel difference thresholds is essential to reduce false

alarms and improve the robustness of test results.

Pixelmatch also includes options for setting `maxDiffPixelRatio` and `maxPixelDiff`, which specify the maximum number or ratio of pixels that the images are allowed to differ while still being counted as identical, along with other configurations. The results are then reported by one of the built-in reporters, which provides the output in the selected format, such as HTML or JSON.

2.4.2 Interaction in Tests

Playwright supports performing actions within tests. To interact with elements on a webpage, they must first be located using the Locators API. A locator is a method used to identify elements on the page. There are several types of supported locators; see Table 2.1:

Locator	Description
`page.getByRole()`	Locates an element using explicit and implicit accessibility attributes.
`page.getByText()`	Locates an element by its text content.
`page.getByLabel()`	Locates a form control using the associated label's text.
`page.getByPlaceholder()`	Locates an input element by its placeholder text.
`page.getByAltText()`	Locates an element, typically an image, by its alternative text.
`page.getByTitle()`	Locates an element by its title attribute.
`page.getByTestId()`	Locates an element using the `data-testid` attribute (configurable to use other attributes).

Table 2.1: Supported Locators in the Locators API

Once the element or elements are found, Playwright waits for the element to become actionable and then performs the action (see Listing 2.2). The most commonly used actions are listed in Table 2.2.

Listing 2.2: Example of an interaction using Playwright.

```
1
2 const getStarted = page.getByRole('link', {
     name: 'Get started' });
3
4 await getStarted.click();
5 });
```

Action	Description
`locator.check()`	Check the input checkbox.
`locator.click()`	Click the element.
`locator.uncheck()`	Uncheck the input checkbox.
`locator.hover()`	Hover mouse over the element.
`locator.fill()`	Fill the form field with input text.
`locator.focus()`	Focus the element.
`locator.press()`	Press a single key.
`locator.setInputFiles()`	Pick files to upload.
`locator.selectOption()`	Select an option from the dropdown.

Table 2.2: The most commonly used Playwright actions

2.5 Mutation Testing

Mutation testing is a technique for determining the fault-detecting capabilities of a test suite [5, 24] [25]. Unlike unit testing, which tests the code's behaviour by providing different input parameters, mutation testing alters the source code. The method aims to emulate errors that can occur naturally by using mutant operators to introduce errors or so-called "mutants" in the code.[25]. The test suite is then run to see if an introduced modification is detected. If it is detected, resulting in the failure of at least one test, the mutant is said to have been "killed." If the test suite kills a high percentage of mutants, it indicates strong fault detection capabilities, assuming the mutant operators used are appropriate [24].

Typically, during mutation testing, the focus is primarily on the logic of the code. Some utilised mutation operators include arithmetic operator replacement, which may involve substituting the subtraction operator (-) for the addition operator (+) in expressions such as `y - x`, and logical connector replacement, which may replace the logical AND operator (&&) with the logical OR operator (||) in expressions like `y && x` [**?**][26].

An example can be found in Figure 2.5, presenting a basic JavaScript function called `isValidUser` before and after it has been mutated. In the original version, the function returns true only if the user is both verified and active. In contrast, the mutated version returns true if either one of those conditions is met. To kill this mutation, tests are needed to ensure that the function returns true only when both requirements are satisfied.

Mutation testing can be conducted using either first-order or higher-order

(a) Original function

```
function isValidUser(user) {
  return user.isActive && user.isVerified;
}
```

(b) Mutated function

```
function isValidUser(user) {
  return user.isActive || user.isVerified;
}
```

Figure 2.5: Comparison of the function isValidUser before and after introducing a mutation

mutations, which differ in the number of mutations introduced to the system under test simultaneously. Traditional mutation testing is considered to be first-order and only uses a single mutation at a time. Higher-order, on the other hand, introduces multiple mutations at the same time [24].

When applying single mutations, each mutant is introduced individually, and tests are executed to assess whether they successfully identify the change. After executing all the tests, a mutation score is calculated, representing the percentage of successfully killed mutants out of the total number of mutants [27] (see Equation 2.1). A test suite is said to be mutant adequate if 100% of the mutants are killed [27].

$$Mutation\ Score = \frac{Mutants\ killed}{Total\ number\ of\ mutants} \tag{2.1}$$

2.6 Related Work

In this section, we summarise previous research, particularly in the fields of visual regression testing and mutation testing. In this section, we also explore how this study differentiates itself from similar studies.

2.6.1 Visual Regression Testing

Web application testing, especially regression testing, faces many challenges with respect to test efficiency and maintainability. Ricca et al. [28] identify three key issues. First, the tests are **fragile** as they break due to minor changes. Second, they are **strongly coupled** and closely tied to the web application being tested, which leads to poor maintainability. Third, the tests are **incomplete** as fully covering all possible user interaction paths in a complex interface is rarely feasible, leading to low test coverage.

The fragility and maintainability of tests are further examined in the Master's dissertation of Fábio Huang [4]. Huang evaluates a range of VRT tools by deliberately introducing faults, such as font size changes and extra padding. The study shows that whole-page screenshot comparisons can often result in false positives, particularly when using pixel-by-pixel comparison methods. Although conducting element-based comparisons may seem like a more robust approach, Huang criticises it, noting that neighbouring elements can influence the results. Therefore, the dissertation calls for improved isolation strategies to mitigate this issue. Additionally, it highlights the significance of automated VRT tests as a key factor in enhancing maintainability.

Yandrapally and colleagues [9] address the challenge of automatically generating regression tests for GUIs. They also identify fragility and maintainability as common issues in VRT. The authors emphasise the high costs associated with maintaining and manually creating regression test suites and the difficulties in developing a robust suite that is tolerant of minor changes that do not impact functionality. Similar to Huang [4], they find that whole-page screenshots can be overly rigid, leading to excessive false positives and fragile test suites. Instead, they propose treating a webpage as a hierarchy of fragments. By breaking the page into smaller segments, their approach aims to improve functional equivalence detection while leveraging state exploration to identify distinct states and reduce near-duplicates. By utilising both structural (DOM comparison) and visual comparison strategies, they successfully detect near-duplicate states while still covering a large portion of the state space. The outcome of their study is a tool called FRAGGEN, which effectively detects visual changes to the UI while remaining tolerant of small, unrelated differences.

With respect to the issue of test completeness in visual regression testing, the

aspect of test coverage has been discussed in numerous studies. Test coverage is in fact not limited to visual regression testing, but affects web application testing as a whole. The problem arises because web applications often consist of many components that are used to compose complex user interfaces. There are typically a significant number of actions that can transition the application from one state to another, leading to an almost infinite number of very similar states. This raises the question of which states or test paths should be selected for testing and how to make those choices. Several approaches have been proposed to address this issue. Some studies recommend a combination of state exploration and user-guided input to identify which test paths should be prioritised [29]. However, other research highlights a significant drawback of this method: it often focuses solely on the "happy path," meaning it tests the system based on how it is intended to function rather than how it behaves in practice [30].

Leotta et al. [31] approach VRT from a DOM-testing perspective. Their work highlights practical obstacles related to constructing visual tests. They list specific problems when generating visual tests. Elements may appear multiple times on the same page, their visual appearance changes across states or sessions, and complex interactive behaviours are involved (e.g., drop-downs). To address these challenges, they create a tool that converts traditional DOM-based tests using DOM locators into tests utilising visual locators and computer vision.

2.6.2 Mutation Testing

Mutation testing is a well-established technique for assessing the effectiveness of test suites [32] [26]. Papadakis et al. argue for the importance of mutation testing, stating that it offers a way to make claims about test effectiveness verifiable. Consequently, the failure to detect specific mutants implies the inability to identify certain types of faults [26].

However, in their study, Olieveria and colleagues [33] highlight the limitations of traditional mutation operators when applied to GUI-level mutation analysis. They demonstrate that standard mutation operators, which generate errors in the source code, have a limited impact on the GUI. Consequently, the study argues that mutation testing must be specifically adapted for GUI testing. To address this, it suggests the introduction of mutation operators designed to

visually affect the GUI. To achieve this, they posit 18 mutation operators that can be divided into three classes, “removing”, “adding,” and “modifying.” The first two classes reference adding or removing functionality to change the behaviour of the system under test (SUT). In this context, this could be removing or adding a component, such as a button, from the GUI. The modifying operators, on the other hand, alter the existing GUI in various ways, such as changing its appearance, repositioning elements, or performing similar modifications. They evaluate the efficiency and applicability of the proposed GUI mutation operators, across two different applications, a randomly selected GUI-based application from an open-source library and a real-world complex application. They generate both traditional and GUI mutation operators for the subject applications, and found that the GUI modification operators demonstrated a fault-seeding effectiveness of approximately 84%, whereas the traditional modification operators exhibited only 12% effectiveness.

This idea of visually-oriented mutation has also been explored in other contexts. Deng et al. [34] introduced a framework for generating GUI mutants in Android applications, focusing on runtime faults and interface anomalies. More recently, Tafreshipour et al. propose Ma11y [35], a novel set of mutation operators aimed at evaluating the accessibility of web pages. Ma11y categorises its 25 mutation operators into four principles of accessibility: perceivable, operable, understandable, and robust. The perceivable operators, for example, alter the visual appearance of web components, including changes to font size, foreground colour, and the removal of visual indicators such as outlines around links to simulate accessibility barriers that affect users with visual impairments. These studies collectively underscore the growing recognition that mutation testing must evolve to reflect the unique challenges of GUI testing.

2.6.3 Novelty Statement

This study builds upon the research conducted by Yandrapally et. al [9]. While they divide a web page into small, stable segments, we propose a different approach by generating tests based on a React Component Library. We argue that a component library functions as a fragmented version of a web page, simplifying the challenging task of page segmentation.

Generating tests for the component library presents several advantages. For

example, it minimises the occurrence of duplicate fragments, as a web page may contain multiple instances of the same element, such as buttons. In contrast, using a component library allows for testing a button in isolation. Additionally, since component libraries are often utilised across multiple products or web pages, testing the components directly can save significant time. This will also reduce the strong coupling between the test suite and the web page, as component libraries remain unaffected by structural changes to the web pages where they are used. Furthermore, testing at the component library level decreases the overall number of tests needed and reduces coupling, which enhances the maintainability of the test suite. This approach is particularly beneficial in large products where fragmenting every page would be time-consuming.

In this study, we use insights from closely related literature to inform our evaluation methodology. These studies provide a valuable foundation for identifying suitable mutation operators to assess the fault-detection capabilities of VRT suites.

Novelty Statement

By shifting the focus from page-level segmentation to component-level testing, our approach not only streamlines test generation but also improves scalability, maintainability, and fault detection, paving the way for more robust and efficient visual regression testing in modern web development.

Chapter 3

Design & Implementation

In this chapter, we outline the technical contributions of the thesis, beginning with an overview of the implementation of AutoVRT. We discuss the details of state exploration and test generation, and conclude with an explanation of the automated mutation testing process.

3.1 Overview of AutoVRT

We develop AutoVRT to automatically generate visual regression tests for React components, emphasising component states that can only be accessed through user interaction. AutoV consists of two phases, Phase I, where new tests are generated, and Phase II where we evaluate the impact of the generated tests (see Figure 3.1).

We achieve Phase I by taking Storybook stories as input and systematically using interactions to find new states. The output of the phase is screenshots of the newly discovered states, along with generated tests that reflect the actions taken to reach these states. These generated tests become the improved VRT suite. In phase II, we then evaluate the fault-detecting capabilities of the original VRT suite compared to the improved VRT suite by using mutation analysis.

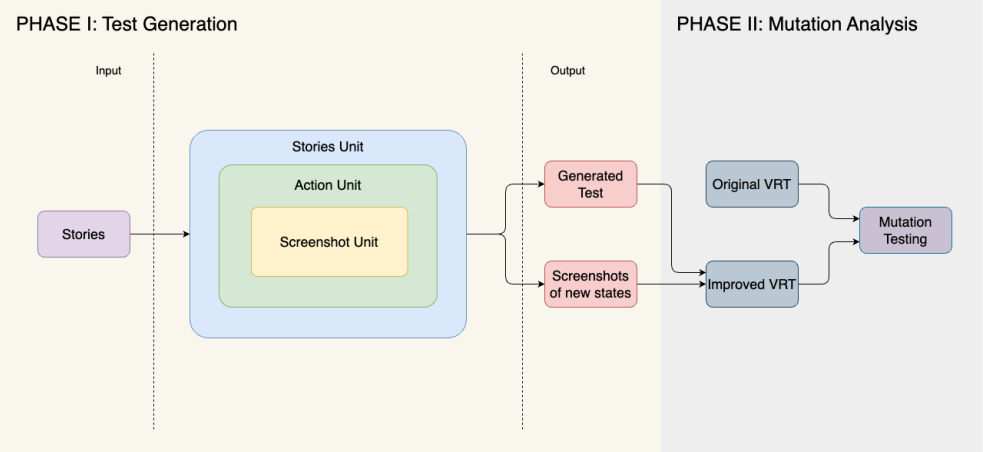


Figure 3.1: A structural overview of AutoVRT.

3.1.1 Phase I: Test Generation

We use a modified Playwright test file to create Phase I of AutoVRT. It follows a three-level structure to interact with Storybook components: a Stories unit, an Action unit and a Screenshot unit (see Figure 3.1). At the highest level, the Stories unit iterates through each Storybook story. Within each story, the second unit, the Action unit, cycles through all possible user actions, such as clicking on or hovering over elements. Finally, the innermost unit, the Screenshot unit, iterates over the specific interactive elements within the component, ensuring that every relevant UI element is tested. In the following sections, we provide more details regarding the implementation of the three units.

3.1.1.1 Stories Unit

The primary purpose of the outermost unit is to iterate through the various stories. A flow chart of the unit can be seen in Figure 3.2. Recall from Section 2.3 that each story contains one component that may consist of several underlying HTML elements, some of which are interactive. For each story, a screenshot of the component is taken, and a test is generated and stored in a folder. These images correspond to the images in the original VRT suite and capture the initial state of each component. These images will later be used as a baseline for the subsequent screenshots. Using Playwright's locators, we

identify interactive elements within the components. If we don't find such elements, the Action and Screenshot units are skipped, and the program moves on to the next story.

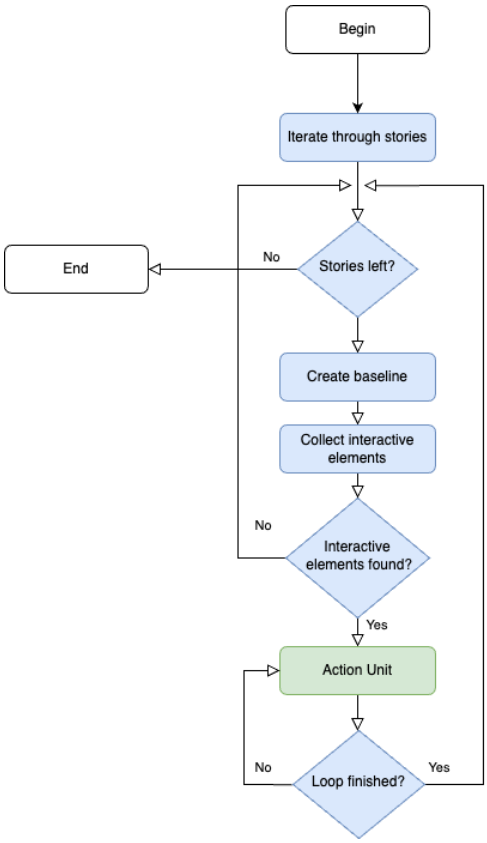


Figure 3.2: Flow chart of the Stories unit

3.1.1.2 Action Unit

If we find interactive elements in the Stories unit, the program progresses to the next unit, the Action unit. It is at this stage that we perform the actual state exploration. We use insights from the component analysis to develop the process. To reach the different states, we need to perform interactions on the components. We use the Locator API from Playwright to collect the interactive elements within these components (see Section 2.4). Next, to interact with the elements, we utilise Playwright's built-in interaction functions. However, this still raises the question of which elements should be interacted with and in what order.

To perform an exhaustive state exploration, we must consider all permutations of actions and elements. Additionally, performing the same action multiple times on the same element could lead to different states, which we also must consider. Because of this complexity, there is no clear stopping criterion. For example, consider a date picker component that allows the user to switch to the next month by clicking on a button. The user can continually click the "next" button such that each month represents a unique visual state with a different month and year. This presents a challenge if all states are examined, as there would be an endless number of states to consider. Therefore, we conclude that a complete exploration is not feasible.

Instead, to avoid this issue, we choose a simpler approach. Since many actions can lead to the same state, we decide to prioritise depth by focusing on a limited number of actions and using them repeatedly rather than considering all available actions. We determine during the component analysis that clicking, hovering, and focusing are the most common actions. This is established by manual analysis of all the components. Therefore, the study is limited to these actions alone. Additionally, we decide to interact with elements sequentially.

The result is an implementation that examines one action at a time, repeatedly utilising the elements associated with that action. The Action unit continues executing as long as there are actions left to perform (see Figure 3.3). Inside the unit, we iterate through the interactive elements within the components, performing the current action and visual comparison (performed in the Screenshot unit). Every time an action is performed, the action and the element on which it was performed are pushed to a stack. Once every element has been interacted with, we check to see if the interaction produces a visual difference for any of the elements. We use Playwright's API for Visual Tests, which utilises the Pixelmatch library [15] to calculate the visual difference. If a difference is detected for at least one element, another iteration of all elements and the same action will be performed. Otherwise, the action is removed, and the loop continues to the next action.

3.1.1.3 Screenshot Unit

The Screenshot unit iterates over the interactive elements (see Figure 3.4). As long as there are elements remaining, it follows these steps: the current action is performed on the selected element. After the action is completed, a

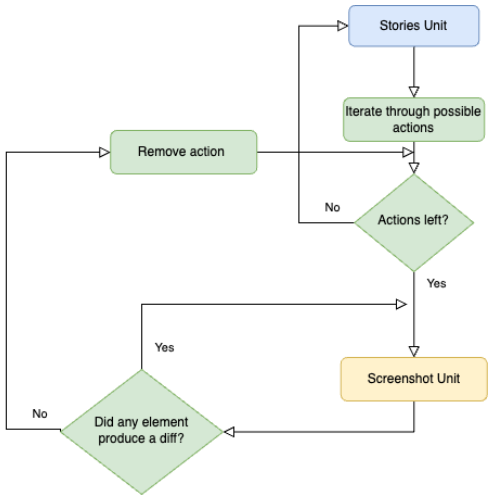


Figure 3.3: Flow chart of Action unit

screenshot of the new state of the page is captured. This screenshot is then compared to the baseline image generated in the Stories unit. If there is a difference between the two images, it indicates that the interaction has resulted in a new state compared to the initial state.

To ensure that the new state has not been visited before, previously taken screenshots from the state exploration are also compared. If the new screenshot does not match any of them, it signifies that a new state has been discovered. The image is then saved, and a test is generated that includes the actions taken to reach the new state.

To generate visual tests that accurately reflect the actions performed during the state exploration, we implement specific functions within AutoVRT to produce the test content. Listing 3.1 presents the `generateTestContent` function that is responsible for producing the content of each generated test. As new states are identified, the generated tests are required to correspond to these states. The general structure of the test (see Lines 8 to 15) remains the same for each test. Furthermore, this correspondence includes the specific actions performed on designated elements (see Lines 18 to 23), which are necessary to reach the particular state. Furthermore, it is essential for the generated tests to align with the saved images, not only to ensure successful execution on subsequent runs but also to verify that the images match the content outlined in the test files.

Listing 3.1: The code used for test generation

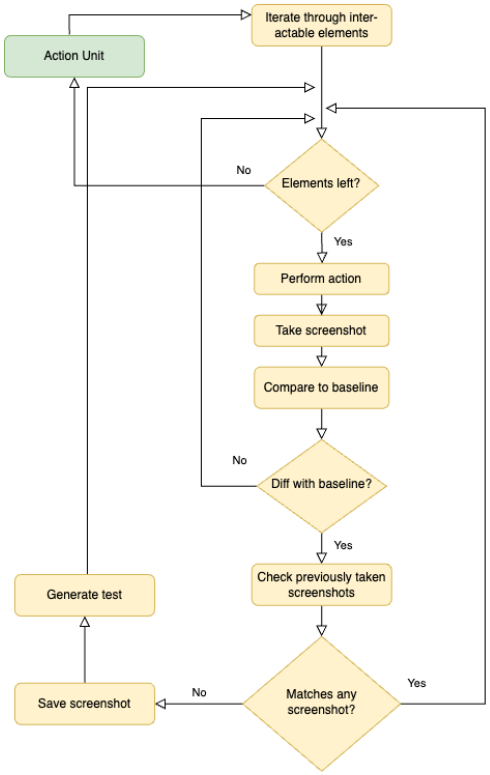


Figure 3.4: Flow chart of the Screenshot loop

```
export default function generateTestContent(
  actionsTaken: any,
  storyId: any,
  image: any,
  params: any,
  locator: any
) {
  let testContent = `
    import { expect, test } from '@playwright/test';\n\n
    test('Test for ${storyId}', async ({ page }) => {\n\n
      await page.goto(\`/iframe.html?${params.toString()}\`); \n
      await page.waitForSelector('#storybook-root'); \n
      await page.waitForLoadState('networkidle');\n
      const sbRoot = page.locator('#storybook-root');\n
      const elements = sbRoot.locator('${locator}'); \n`;


  if (actionsTaken) {
    actionsTaken.forEach((action: any) => {
      testContent += `
        await elements.nth(${action.idx}).${action.action.name}();\n`;
    });
  }
  const pathParts = image.split('/');
  const formattedPath = `[${pathParts.map((item) => `'${item}'`).join(', ')}]`;

  testContent += `
    await expect(page).toHaveScreenshot( ${formattedPath},{
      fullPage: true,
      animations: 'disabled'});`;
  testContent += `});\n`;

  return testContent;
}
```

3.1.2 Phase II: Mutation Analysis

We leverage mutation testing to assess the fault-finding capabilities of the original VRT suite in comparison to the VRT suite that includes the additional tests generated by AutoVRT. This process begins with the creation of CSS files, each containing a single mutant. These files are then used to perform first-order mutation testing on both the original VRT suite and the version that includes the additional generated tests.

AutoVRT conducts mutation analysis in two distinct stages, utilising two Python scripts. The initial script generates single mutations within the CSS files. This script accepts the CSS associated with a specific component and makes slight alterations to produce a mutated variant. The mutation operators are delineated using regular expressions (regex). For each mutator, the script systematically parses the file, identifying matches and generating a new file that incorporates the specified substitutions. Consider an example presented in Listing 3.2 where we can see an excerpt from the CSS file of an Avatar component. In this listing, two colour-related properties are found on Lines 9 and 10. If we want to substitute these, the script would produce two new files, as seen in Listings 3.3 and 3.4, where the two properties are replaced with a new value.

Listing 3.2: Original CSS code of the Avatar component before the first mutation script

```
1  .#{$prefix}avatar {
2    position: relative;
3    display: inline-block;
4    width: $avatar-size;
5    min-width: $avatar-size;
6    height: $avatar-size;
7    min-height: $avatar-size;
8    margin-right: $space-small;
9    color: $avatar-text-color;
10   background: $avatar-bg-color;
11   ...
```

Listing 3.3: Mutated version 1 of the Avatar CSS

```
1  .#{$prefix}avatar {
2    position: relative;
3    display: inline-block;
4    width: $avatar-size;
```

```
5    min-width: $avatar-size;
6    height: $avatar-size;
7    min-height: $avatar-size;
8    margin-right: $space-small;
9    color: blue;
10   background: $avatar-bg-color;
11   ...
```

Listing 3.4: Mutated version 2 of the Avatar CSS

```
1  .#{$prefix}avatar {
2    position: relative;
3    display: inline-block;
4    width: $avatar-size;
5    min-width: $avatar-size;
6    height: $avatar-size;
7    min-height: $avatar-size;
8    margin-right: $space-small;
9    color: $avatar-text-color;}
10   background: blue;
11   ...
```

The second script leverages the hot reloading feature of Storybook. This means that if alterations are made to the CSS file, the page will update automatically without recompiling the project. This allows the script to substitute the original CSS files one at a time, run the VRT suites, and note the result. In the preceding example, the script would replace the Avatar CSS with the mutated versions one at a time, running both test suites after each substitution. In order to get the results of multiple test runs in the same place, AutoVRT uses a custom reporter to report the results of the test, along with the mutation that was used, in a CSV file.

3.1.3 End-to-End Example

If we consider the Switch component, which has four different stories: "Standard", "Disabled", "Subtle", and "Label on the left", AutoVRT works the following way:

3.1.3.1 Phase I

In Phase I, AutoVRT begins in the Stories Unit by iterating through each Storybook story one at a time, starting with the first one in the list, for example, the “Standard” version of the Switch component. The first step is to create a baseline for the component by capturing a screenshot of its initial state (see Figure 3.5). This screenshot serves as a reference point for future comparisons. At this stage, a test is generated that navigates to the component’s page and takes a screenshot, without performing any interactions (see Listing 3.5).

Figure 3.5: The initial state of the Switch component

Listing 3.5: The baseline test for switch standard

```
1  import { expect, test } from '@playwright/test';
2
3  test('Test for components-switch--standard',
       async ({ page }) => {
4    await page.goto(`/iframe.html?id=components-
       switch--standard&viewMode=story`);
5
6    ...
7
8    await expect(page).toHaveScreenshot(
9      [
10       'interaction-screenshots',
11       'interaction-screenshot-components-switch
           --standard.png',
12     ],
13     {
14       fullPage: true,
15       animations: 'disabled',
16     }
17   );
18 });
```

Next, AutoVRT looks for any interactive elements within the component. In the Switch case, it detects two: the switch element itself, which is typically an HTML input, and the associated label. If no interactive elements are found at

this point, the component is considered static and AutoVRT moves on to the next story. But since there are interactive elements here, the process continues into the Action unit.

The Action unit sets the current action to "click" and passes control to the Screenshot unit, where this action is applied to each interactive element, one at a time. The first element interacted with is the label. Clicking the label changes the state of the Switch, it becomes checked (see Figure 3.6). AutoVRT captures a screenshot of this new state and compares it with the baseline. Since there is a visible difference, it recognises this as a new state, saves the screenshot, and generates a test that includes a click on the label.

Figure 3.6: The state of the Switch after clicking the label

Listing 3.6: The generated test that clicks the label of the Switch component

```
1  import { expect, test } from '@playwright/test';
2
3  test('Test for components-switch--standard',
       async ({ page }) => {
4    await page.goto(`/iframe.html?id=components-
       switch--standard&viewMode=story`);
5
6    ...
7
8    await elements.nth(0).click();
9
10   await expect(page).toHaveScreenshot(
11     [
12       'interaction-screenshots',
13       'interaction-screenshot-components-switch
           --standard.png',
14     ],
15     {
16       fullPage: true,
17       animations: 'disabled',
18     }
19   );
20 });
```

The next interaction is with the Switch itself. Clicking it unchecks the Switch but leaves it focused, which results in a visual indicator, a teal border appearing around the element (see Figure 3.7). A new screenshot is taken and compared first with the baseline and then with the previously saved screenshot from the label interaction. Since it's a unique visual state, another test is generated, this time including both the click on the label and the subsequent click on the Switch (see Listing 3.7).

Figure 3.7: The state of the Switch after clicking the label and then the switch

Listing 3.7: The generated test that clicks the label and the switch of the Switch component

```
1  import { expect, test } from '@playwright/test';
2
3  test('Test for components-switch--standard',
       async ({ page }) => {
4    await page.goto(`/iframe.html?id=components-
       switch--standard&viewMode=story`);
5
6    ...
7
8    await elements.nth(0).click();
9    await elements.nth(2).click();
10
11   await expect(page).toHaveScreenshot(
12     [
13       'interaction-screenshots',
14       'interaction-screenshot-components-switch
           --standard.png',
15     ],
16     {
17       fullPage: true,
18       animations: 'disabled',
19     }
20   );
21 });
```

Once all the elements have been interacted with for the current action (in this case, click), AutoVRT checks whether any of those interactions led to new

visual states. Since they did, it repeats the same action (click) on all elements again. This allows it to uncover new states that may emerge only after certain sequences of interactions. After a third round of clicks, no new states are discovered, so AutoVRT moves on to try the next interaction type, hover and then focus, applying the same exploration and comparison steps for each.

If no additional visual differences are detected from those actions, AutoVRT concludes that all possible states for this component and its current story have been explored. It then returns to the Stories unit and continues with the next story in the list, repeating the process.

The other stories produce similar results, generating two tests per component (not including the baseline), except for the disabled version, which has no interactive elements and thus only the initial state is captured.

3.1.3.2 Phase II

In phase II, the first Python script analyses the component's CSS file. It searches for matches to a specified regex, such as one that targets various colour properties. In Listing 3.8, we can see two out of the total twelve matches to this regex (see Line 16 and 30). The script will generate twelve new files, each introducing a mutation in the colour properties by altering the values of the matched properties to different ones.

Listing 3.8: The CSS of the Switch compoennt

```
1  .#{$prefix}switch {
2    display: inline-flex;
3    height: $switch-height;
4
5    margin-right: $space-small;
6    margin-bottom: $space-related;
7    user-select: none;
8
9    ...
10
11     + label {
12       position: relative;
13       display: inline-block;
14       height: $switch-height;
15       margin-right: 0;
16         color: $text-color ;
```

```
17        font-size: $font-size;
18        font-weight: $font-regular;
19        line-height: $switch-height;
20        vertical-align: middle;
21        outline: none;
22        cursor: pointer;
23        width: $switch-width;
24
25        &::before {
26          content: '';
27          display: inline-block;
28          height: $switch-height;
29          width: $switch-width;
30            background:  $gray-5; ;
31          ...
32
33      }
34  }
```

Once all twelve mutated CSS files for the Switch component have been generated, we proceed to use the second script. This script will substitute each mutated CSS file one at a time and then run both the original VRT suite and the tests generated in phase I after each substitution. This process will help us assess the fault-detection capabilities of both the original VRT and the generated tests.

Chapter 4

Experimental Methodology

This chapter outlines how we evaluate AutoVRT on a React component library. We start by presenting and motivating the research questions, then describe the study subject and the experimental setup. Finally, we present the protocol used to address the research questions.

4.1 Research Questions

- **RQ1: What types of mutants are most relevant to VRT in terms of their prevalence and effectiveness in evaluating VRT suite quality?**

  In order to effectively test the fault-detecting capabilities of the VRT suites, we want to identify the subset of visual mutants that would be suitable.

- **RQ2: What is the current mutation score of the original VRT suite?**

  To determine how effectively the tests generated by AutoVRT improve the mutation score, we must first calculate the mutation score of the original VRT suite.

- **RQ3: What are the most common UI actions for the React Component Library?**

  Since there are no tests involving interactions, we want to identify the common actions that create impactful tests. This analysis aims to

enhance our understanding of the complexity of the components, the various states they can enter, and the actions that lead to these states. The findings from this analysis will guide us in selecting which actions to use during the state exploration.

- **RQ4: How effectively can automated visual regression test generation using state exploration enhance the mutation score of a VRT suite?**

  We will compare the mutation score of the VRT with the generated test to the score without it to investigate the impact of the generated tests.

4.2 Study Subject

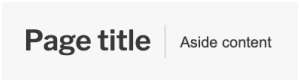
The study subject in this thesis is the OSTTRA React component library, which is a frontend library developed and actively maintained by OSTTRA. It is used by more than ten product teams, all of whom rely on the components to build robust user interfaces for their products. Reliability is a key expectation from these teams when utilising the library. The component library is built collaboratively by the product teams, where many developers are active in the code base over time. The library consists of 50 components with different complexities, ranging from basic components to more complex components containing multiple subcomponents.

The basic components are non-interactive and consist of a single state. Some examples of these components can be seen in Figure 4.1. One example is the Tag component (see 5.2a), which is used in filter functions to display selected filters. Another example is the Avatar component (see 5.2b), which shows the active user. The Aside component (see 4.1c) is used to place content next to the title on certain pages. Additionally, there is a Container component (see 4.1d) that displays information in a box with a white background and a LabelValue component (see 5.2c) that shows simple labels along with their corresponding values.

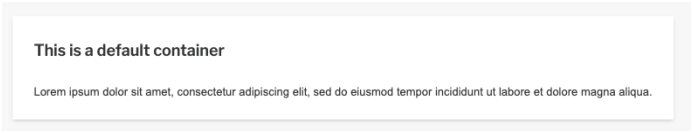
The component library also contains several components of increased complexity, such as the Accordion component. This component comes in several variants, but the standard Accordion consists of a clickable label with a chevron (see Figure 4.2a). When clicked, the hidden content unfolds (see Figure 4.2b). In this category, we also find Buttons, Text Areas, Number

(a) Tag component.

(b) Avatar component.



(c) Aside component.



(d) Container component.

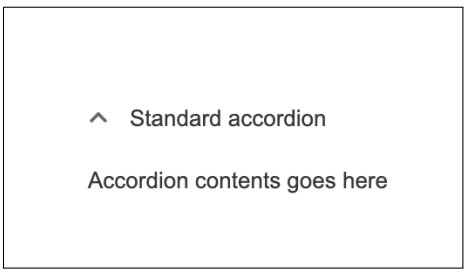
(e) LabelValue component.

Figure 4.1: Displaying examples of one state components from OSTTRA's component library.

Inputs, and similar components. Because these components are interactive, they possess multiple states.

The most complex components consist of several subcomponents and often involve more advanced interactions and dynamic behaviours. For instance, a Table component includes multiple buttons and text fields and allows the user to perform complex behaviour patterns.

(a) Initial state of an Accordion component.



(b) State of an Accordion component after it was clicked.

Figure 4.2: Depicting the initial state and the state after interaction for an Accordion component from OSTTRA's Component library.

Table 4.1 presents a list of 50 components in the component library and categorises them as either static or interactive. For this study, we select five components from the library to represent the entire suite. We excluded components with only one state, which are the static ones, as they do

not exhibit any interactions. This left us with 38 components to choose from. Additionally, we do not select components that contain multiple subcomponents, as testing them would yield the same information as testing their individual subcomponents. This category includes components such as the Datepicker, Monthpicker, and Table. From the remaining 29 components, we randomly selected five for analysis: Accordion, Checkbox, Dropdown, Radio, and Switch.

Static Components	**Interactive Components**
Aside	Accordion
Avatar	Anchor
Badge	BackToTop
Comments	Breadcrumbs
Container	Button
EmptyState	Checkbox
Filtership	Datepicker (Cyclic)
Flag	Dialog
Highlight	Drawer
Icon	Dropdown
Inactivity Overlay	FilterSteps
LabelValue	InfoBubble
List	Monthpicker (Cyclic)
Loading	Navigation
Notification	NumberInput
Staffbox	Paginator
Tag	Progress
	Radio
	Select
	Slider
	Steps
	Switch
	Table
	Tabs
	Textarea
	TextInput
	Toast
	Tooltip
	UserFeedback

Table 4.1: OSTTRA React component library: Static vs Interactive

4.2.1 Experimental Setup

The setup of this thesis includes the OSTTRA React component library, the original VRT suite, and AutoVRT created in TypeScript. The automated mutation analysis phase within AutoVRT is implemented in Python.

4.2.1.1 Original VRT suite

The test suite consists of a single test file and utilises Storybook for testing. It iterates through each story (a predefined variant of a component) and tests them individually. For each test, Playwright navigates to the corresponding Storybook page, as each component has its own dedicated page. As detailed in Section 2.4, once the page has loaded, Playwright captures a screenshot of the component using the function `await expect(page).toHaveScreenshot()`. The function searches for a previously saved screenshot of the story. If one exists, the function will visually compare the old and newly captured image. Otherwise, a screenshot will be captured and saved to be used as a new baseline. The screenshots captured by the test are 1280 x 720 images in PNG format. The screenshots depict the components in their initial state, such as the Accordion component in Figure 4.2a.

4.3 Protocol

In this section, we outline the protocol to address the research questions presented in Section 4.1. To answer the questions, we use the five selected components from the OSTTRA React component library.

4.3.1 RQ1: Mutation Operators for VRT suites

We consult relevant literature [33][35] to identify the most relevant mutants for the VRT test suite. Insights gained from the literature are combined with manual mutation analysis. The identification of these relevant mutants is crucial for the VRT, as it enables us to effectively evaluate the fault-detection

capabilities of the VRTs. By using these mutants, we can simulate potential faults and examine how well the VRTs identify and handle these issues. This provides us with valuable feedback on the effectiveness of our testing strategies and helps us improve the reliability of the software under evaluation.

4.3.2 RQ2: Mutation Score of the Original VRT suite

In Phase II of the implementation, we focus on calculating the mutation score of the original VRT suite by utilising various components from the component library. The mutation score is a critical metric that indicates how effectively the VRT suite can detect faults, essentially measuring its effectiveness in identifying changes or errors in the visual output of applications. The mutation score derived from this assessment serves as a baseline. This baseline will enable us to evaluate and compare the fault-detection capabilities of the original VRT against subsequent improvements we implement.

4.3.3 RQ3: Selecting Actions for Test Generation

We conduct a thorough analysis of the components to identify the most common actions within the OSTTRA React component library. We review the source code of the components, examining both the TypeScript implementation and the associated CSS files to understand the behaviour exhibited by each component. Additionally, we interact with the components using Storybook. These insights are essential in order to create AutoVRT that systematically explores the components and generates new tests.

4.3.4 RQ4: Comparative Analysis Between the Original and Improved VRT Suite

To address RQ4, we utilise the tests generated in Phase I of AutoVRT. This becomes the improved VRT. After phase I is completed, we calculate the mutation score for the improved VRT in Phase II. We then compare the newly obtained results with the previously calculated mutation score of the original VRT to evaluate the effectiveness of the improved VRT. This comparison helps

validate whether AutoVRT adds value to the original test suite with respect to its fault detection capability.

Chapter 5

Experimental Results and Analysis

In this chapter, we present the results of the thesis per our research questions. We begin by discussing findings related to what constitutes a suitable mutant operator. Next, we present the results from the analysis of the OSTTRA React Component Library. Finally, we compare the results of the mutation analysis for both VRTs.

5.1 RQ1: Mutation Operators for VRT suites

The initial research question focuses on identifying appropriate mutation operators for determining the fault-detection capabilities in VRTs. To achieve this objective, we conduct manual mutation analysis on several components to identify potential candidate mutation operators. Our findings indicate that traditional mutant operators, in most instances, do not produce visual differences, which supports the findings presented in the existing literature [33]. Consequently, we draw inspiration from similar studies that use the alternative strategy of modifying the visual properties instead [33], [35][34].

We use manual investigations to determine if Playwright is sensitive to the CSS properties in different mutants. We establish that Playwright can distinguish all mutants of the most common CSS properties, including variations in colour,

padding, and margin. This suggests that the specific property does not impact its detection. However, as a regression can appear anywhere in the component, we find that a suitable mutation operator must target as many parts of the components as possible to provide a good estimation of the effectiveness of the VRT suite.

During our analysis of the code and manual testing, we notice that the most frequently occurring CSS property across all components is some form of colour (see Table 5.1). In the table, we list the frequency of the most common CSS properties in the OSTTRA React component library. The colour properties, "background," "color," "background-color," and "border-color," are highlighted in yellow. We see that the total number of occurrences of these colour properties is 684, which far exceeds that of any other properties.

Property	**Frequency**
background	295
color	245
width	258
display	192
height	181
margin-right	95
background-color	88
position	134
font-size	56
border-color	56

Table 5.1: The 10 most common CSS properties and their frequency in the OSTTRA React component library. Highlighted rows represent colour-related properties.

We also find that all components include some kind of colour property. Even the components with only one state possess several instances. For example, the Avatar component, which consists of only a circle with initials, has five occurrences of some kind of colour (see Listing 5.1). Often, content that can be expanded or interacted with includes some variation of a colour property for text, background, or element. Consequently, we adapt our Regex to match different CSS colour properties such as "background-colour" and similar variations. Although we could utilise other properties or a combination of multiple properties as mutation operators, manual testing reveals that using more than one mutation operator does not enhance fault-detection; it only increases execution time.

Listing 5.1: The CSS of the Avatar component. The occurrences of colour-related properties are highlighted in grey

```
@import '../../partials/variables';
@import '../../partials/mixins';

.#{$prefix}avatar {
  position: relative;
  display: inline-block;
  width: $avatar-size;
  min-width: $avatar-size;
  height: $avatar-size;
  min-height: $avatar-size;
  margin-right: $space-small;
  color: $avatar-text-color;
  background: $avatar-bg-color;
  ...
  &.#{$prefix}primary {
    background: $teal;
  }

  &.#{$prefix}staff {
    background: $staff-color;
  }
  ...
  .#{$prefix}icon {
    position: absolute;
    inset: 0;
    margin-right: 0;
    font-family: $font-icon;
    line-height: $avatar-size;

    &::before {
      color: $avatar-text-color;
      font-size: 14px;
      line-height: inherit;
    }
  }
}
```

RQ1: Summary of Results

- CSS-based **mutation operators** are used instead of traditional mutation operators, inspired by related studies.
- Manual analysis confirms that **Playwright can detect visual changes** from mutants involving common CSS properties like `color`, `background`, `padding`, and `margin`.
- **Colour-related properties** are the most frequent across the component library, with a total of **684 occurrences**.
- A simplified mutation operator focusing on colour-related CSS is sufficient. Using multiple operators does **not improve fault detection**, but **does increase execution time**.

5.2 RQ2: Mutation Score of the Original VRT suite

Per RQ1, we identify 59 instances in the CSS of the five components that include some form of colour, enabling us to introduce mutants. In Table 5.2, we present the number of stories, the number of mutants introduced into the CSS, and the mutation score per component as well as in total. For example, in the first row, we can see that the accordion has six stories, meaning there are six alternative versions. We introduced five mutations, indicating that five instances of colour properties can be found in the CSS. The test suite achieved a mutation score of 0.20, which means that 20% of the mutations were detected.

The number of mutants we introduce per component ranges from 5 (for the Accordion) to 21 (for the Dropdown), with an average of 11.80 mutants per component (see Table 5.2). We conduct first-order mutation analysis to calculate mutation scores for the original VRT. The mutation scores range from 0.10 to 0.43, with an average score of 0.28 per component. The Dropdown component has the lowest initial mutation score at 0.10, while the Radio component has the highest score at 0.43.

An example of a mutant that can be detected by the original VRT is depicted in Figure 5.1. This mutation changes the colour of the caret in the Dropdown

Component	# Stories	# Mutants	Mutation Score
Accordion	6	5	0.20
Checkbox	7	15	0.40
Dropdown	4	21	0.10
Radio	6	7	0.43
Switch	4	12	0.25
Total	**27**	**59**	**0.22**
Average	**5.40**	**11.80**	**0.28**

Table 5.2: The number of stories, mutants and the corresponding mutation scores for each component of the original VRT.

(a) Expected appearance of the disabled Dropdown.

(b) Dropdown component with mutant.

Figure 5.1: Showing a mutant that alters the colour of the Dropdown caret.

component from dark grey to blue. We can see this mutation on Line 31 of Listing 5.2. However, when we introduced the same colour substitution on Line 17 (instead of Line 31), the mutation goes undetected. While both Line 31 and Line 17 affect the colour of the caret, the former alters it when the component is disabled, whereas the latter does so when the component is hovered over. Since the original VRT does not include interaction, the mutant remains undiscovered.

Listing 5.2: The CSS of the DropDown component. Relevant lines are highlighted in grey

```
1  @import '../../partials/variables';
2  @import '../../partials/mixins';
3  @import 'dropdown-validation';
4
5  .#{$prefix}dropdown {
6    position: relative;
7    display: inline-block;
8    min-width: $dropdown-min-width;
9    margin-right: $space-related;
10   margin-bottom: $space-related;
11   vertical-align: top;
```

```
12
13   ...
14
15     &:hover {
16       &::after {
17         border-top-color:  $dropdown-caret-hover-color;
18       }
19     }
20
21     &.#{$prefix}primary,
22     &.#{$prefix}negative,
23     &.#{$prefix}staff {
24       &::after {
25         border-top-color: $white;
26       }
27     }
28
29     &:disabled {
30       &::after {
31         border-top-color:  blue;
32       }
33     }
34     ...
35
36     }
```

RQ2: Summary of Results

- A total of **59 colour-related CSS mutants** were introduced across five components.
- Mutants per component ranged from **5 (Accordion)** to **21 (Dropdown)**, with an average of **11.80**.
- The initial mutation scores for the original VRT ranged from **0.10 to 0.43**, averaging **0.28**.
- Some mutants are successfully detected, but certain mutations that require interaction go undetected, highlighting the limitations of the VRT's coverage of interaction states.

5.3 RQ3: Selecting Actions for Test Generation

The analysis of the OSTTRA React component library reveals that the components vary significantly both from each other and among the different versions of the same components (the different Storybook stories). Some components do not support interaction at all and therefore only have an initial state (see Figure 5.2).

(a) Tag component (b) Avatar component (c) LabelValue component

Figure 5.2: Examples of single-state components from the OSTTRA React component library.

Most components, however, allow some form of interaction, and a common feature is a border indicating that a component is selected and in focus (see Figure 5.3). In the figure, the initial state of a TextInput is shown, followed by an image of the component with a teal border when the component is selected. This effect is achieved by setting the border-color property on Line 32.

(a) Initial state of a TextInput. (b) TextInput when in focus.

Figure 5.3: Demonstrates how a visual indicator signals when the focus is on the textInput component.

```
1 @use 'sass:color';
2 @import '../../partials/variables';
3 @import '../../partials/mixins';
4 @import '../../partials/icons/variables';
5 @import 'input-validation';
6
7 .#{$prefix}input {
8   display: inline-block;
```

```
9     margin-right: $space-related;
10    margin-bottom: $space-related;
11    vertical-align: top;
12
13    &:last-child {
14      margin-right: 0;
15    }
16
17    .#{$prefix}input-wrapper {
18      background: $input-bg-color;
19      border: 1px solid $input-border-color;
20      border-radius: $input-radius;
21      width: 192px;
22      display: flex;
23      ...
24
25      &:not(:disabled) {
26        &:hover {
27          border-color: $input-hover-border-color;
28        }
29
30        &:focus,
31        &:active {
32          border-color: $input-hover-border-color;
33          box-shadow: $input-shadow;
34        }
35      }
36      ...
37
38  }
```

It is also typical for interactions to trigger something to pop up or out, such as the content of an Accordion (see Figure 4.2). While these behaviours are the most prevalent, there are other types as well. Users can reach these states through various actions such as by using the tab key on the keyboard, or by hovering or clicking. The table presented in Table 5.3 shows the percentages of components supporting various actions.

Clicking and hovering are the most common actions, supported by 70% of the components in the OSTTRA React component library. This represents a significant proportion, particularly when considering that some components lack any interactions. Following clicking and hovering, keyboard actions such as Tab, Escape, and text input emerge as the next most prevalent actions. However, all of these actions require focus, which makes focus the third most

Interaction Type	# Components	Percentage (%)
Mouse Click	35	70
Mouse Hover	35	70
Keyboard Tab	16	32
Escape key	8	16
Keyboard text input	7	14
Space key	8	16
Arrow keys	7	14

Table 5.3: Interaction support among components in the OSTTRA library, including percentage relative to the total number of components.

common action in the library.

In table 5.4, we can see a list of the supported actions for the five selected components from the OSTTRA React component library. All of the components support click, hover and some interaction that requires focus.

Component	Supported Interactions
Accordion	Click, Hover, Tab, Enter, Space
Checkbox	Click, Hover, Space
Dropdown	Click, Hover, Tab, Keyboard text input
Radio	Click, Hover, Space
Switch	Click, Hover, Tab, Enter, Space

Table 5.4: Supported interactions per component

RQ3: Summary of Results

- The OSTTRA React component library includes both static and interactive components, with varying behaviours across different Storybook stories.
- A common interaction feature is a visual indicator (e.g. a teal border) when a component gains focus.
- Many components trigger state changes upon interaction.
- The most supported actions are **clicking and hovering** (each supported by 70% of components).
- **Keyboard interactions are limited**, only 32% of components support Tab navigation, and fewer support Escape, Space, or arrow key interactions.
- All keyboard-based actions require the component to first be in focus, making **focus handling** a critical feature.

5.4 RQ4: Comparative Analysis Between the Original and Improved VRT Suite

We calculate the mutation score for the VRT using the generated tests. In Table 5.5, we summarise the number of tests generated, mutants, and mutation scores both before and after introducing these tests into the VRT suite. By exploring state transitions, we identify 86 new distinct states (not including the initial states) across the five components, leading to the generation of 86 new tests for the VRT suite. The number of tests generated varies significantly across different components, ranging from 6 to 36, with an average of 17.20 new tests per component.

When comparing the mutation score of the original and the improved VRT suite, all components show improvement (see Table 5.5 and Figure 5.4). Overall, the mutation score for the VRT suite improves from 0.22 to 0.53. The component with the highest mutation score before and after is the Radio component, which has scores of 0.43 and 0.86, respectively. The lowest

mutation score and improvement is observed in the Dropdown component, where the score increases from 0.10 to 0.19, meaning that the VRT suite with the generated tests identifies only one additional mutant compared to the original test suite. In contrast, the Switch component shows the largest increase, reaching a score of 0.75 after the generated tests are introduced. On average, the components see an improvement of 130%.

Component	# Mutants	MS Before	MS After	# Generated Tests
Accordion	5	0.20	0.50	18
Checkbox	15	0.40	0.80	36
Dropdown	21	0.10	0.19	6
Radio	7	0.43	0.86	21
Switch	12	0.25	0.75	6
Total	**59**	**0.22**	**0.53**	**86**
Average	**11.80**	**0.28**	**0.62**	**17.20**

Table 5.5: Number of mutants and generated tests, along with mutation scores, for each component before and after integrating the generated tests into the VRT suite.

Recall from Chapter 3 that each test in the original VRT has a corresponding baseline test in the improved VRT. Therefore, the original VRT suite will not detect any mutants that the improved version does not catch. However, to provide a clearer overview of how the static tests compare to the interactive tests in detecting mutants, a Venn diagram is presented (see Figure 5.5). The diagram illustrates the number of detected mutants categorised by components that were eliminated by either interactive or static tests. It's important to note that static tests are included in both test suites, while interactive tests are only found in the improved test suite. The diagram shows that while the static tests account for a significant portion of the detected mutants $\approx 44\%$, while the interactive tests performed even better at $\approx 71\%$

An example of a mutation detected by the newly generated test but not by the original tests can be seen in Listings 5.3 and 5.4. This mutation changes the text colour of the Accordion component's text content from black to blue (see Figure 5.6). This change can only be visually identified when the component is in a hovered state, and therefore, only the generated tests that cause this behaviour can catch it. One of the tests generated by AutoVRT that kills this specific mutant is listed in Listing 5.5. In the test, the label of the Accordion

Figure 5.4: Mutation scores for the VRT before and after the addition of generated tests.

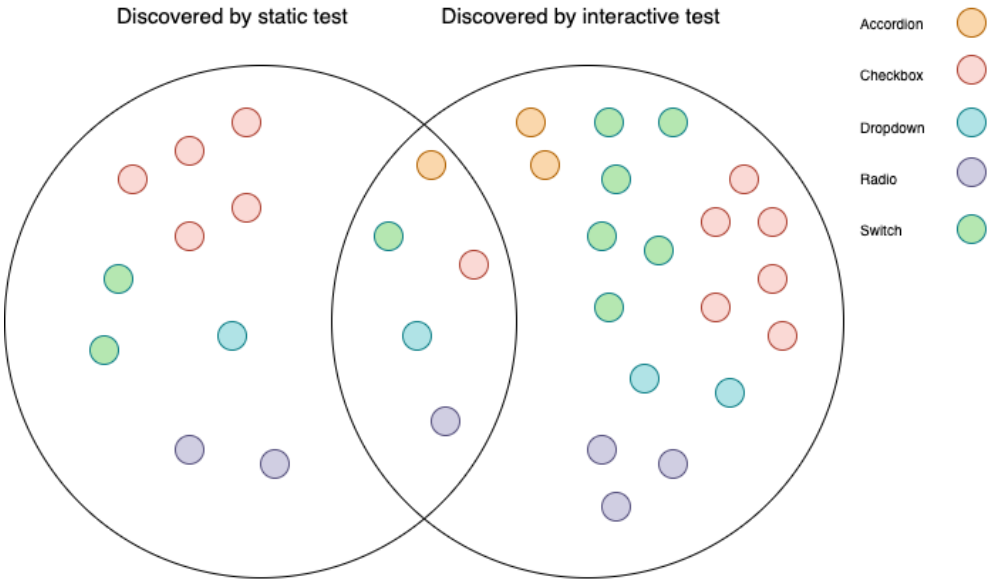


Figure 5.5: Venn diagram of detected mutants

component, which in this case is element zero, is clicked twice (see Lines 11 and 13). This first opens the accordion and then closes it again, leaving the focus on the label and having the same effect as hovering over the label. These actions thus showcase the mutant, allowing the generated test to kill it.

Listing 5.3: Original CSS

```
1 &:hover {
2     color: $black;
3     ...
4 }
```

(a) State of the Accordion component without interaction.

Listing 5.4: CSS where the color property has been mutated

```
1 &:hover {
2     color: blue;
3     ...
4 }
```

(b) State of the Accordion component when the focus is on it.

Figure 5.6: Code and visual states of the Accordion component.

Listing 5.5: Generated test that killed the mutation

```
1
2  import { expect, test } from '@playwright/test';
3
4  test('Test for components-accordion--standard',
       async ({ page }) => {
5    await page.goto(
6      `/iframe.html?id=components-accordion--
         standard&viewMode=story`
7    );
8
9    ...
10
11    await elements.nth(0).click();
12
13   await elements.nth(0).click();
14
15   await expect(page).toHaveScreenshot(
16     [
17       'interaction-screenshots',
18       'interaction-screenshot-components-
           accordion--standard-1744620097225-
           chromium-darwin.png',
19     ],
20     {
```

```
21          fullPage: true,
22          animations: 'disabled',
23        }
24    );
25  });
```

Even though the mutation scores improve for all components, some mutants are not detected by either the original or the improved VRT. One example of this is a mutant that affects the colour of the Switch component when it is both checked and disabled. This indicates that the component must either be set as checked and disabled from the beginning or have its state changed to disabled after being checked. Figure 5.7 illustrates this mutation, showing how the Switch component is expected to appear when it is disabled and checked simultaneously and how it looks when the mutant is introduced. As there was no story with this combination of properties, neither the original VRT nor the improved VRT is able to detect it.

(a) Original appearance of the Switch component when checked and disabled simultaneously.

(b) Mutated appearance of the Switch component when checked and disabled simultaneously.

Figure 5.7: Example of mutants not detectable by the VRTs.

RQ4: Summary of Results

- Through state exploration with AutoVRT, we identify **86 new distinct states**, resulting in 86 additional tests for the VRT suite across five components.
- Mutation scores improved for all components, increasing the overall score from **0.22 to 0.53**.
- **Switch** saw the largest gain (from 0.25 to 0.75), while **Dropdown** showed only a minor improvement (0.10 to 0.19).
- The generated tests detect mutants requiring interaction (e.g., hover effects) which are missed by the original suite.
- Some mutants still remain undetected by both VRTs.

Chapter 6

Discussion

In this chapter, we analyse the results of the thesis in the order of the phases in the AutoVRT. We start by discussing state exploration for test generation, followed by an examination of the generalisability of mutation operators. Finally, we present a comparison between the original VRT suite and the improved version, with a focus on their fault-detecting capabilities. We also explore the reasons behind the observed results, highlighting both the strengths and limitations of AutoVRT.

6.1 State Exploration for Test Generation

Several factors affect state exploration in the testing tool. On the positive side, the process is sequential, actions are performed in a specific order, which leads to deterministic results and minimises variability in each run. However, this approach also has limitations. The tool's implementation dictates a fixed sequence of actions, which may overlook potential states that require a different order to reach. The process consists of first clicking, then hovering, and finally focusing. If a component's state can only be accessed by hovering before clicking, AutoVRT will not be able to identify it.

One aspect is that the test generation AutoVRT is designed to interact only with the first layer of the user interface. In other words, while it can trigger elements that reveal additional content, such as dropdowns, accordions, or expandable

sections, it does not continue interacting with the newly revealed elements. This design choice was intentional, primarily to avoid the risk of infinite interaction loops. However, this also means that certain mutants residing in deeper layers of interaction may go undetected. If a mutant only becomes visually apparent after multiple interactions or within nested UI components, it will be missed by the generated tests.

The interaction order of different elements can similarly result in missed states when they are interacted with differently. However, it can be argued that most variations of the state are often functionally equivalent to those being tested. Thus, the actual value added by accounting for all permutations may be limited. Considering all possible interaction permutations would not only prolong the execution time required to find states but also introduce delays during the regular development process. Therefore, a clear stop criterion must be established; otherwise, this could result in an infinite number of tests. Furthermore, accounting for all permutations could lead to an unmanageable number of tests, potentially redundant, which would negatively impact the maintainability of the test suite.

An alternative approach could involve randomising the order of actions or the sequence of elements. However, this would result in significant variability in tests each time they are run, potentially impacting their reliability since developers might be uncertain about what is being tested during each run. Another possible solution could involve adding an additional phase to AutoVRT that compares tests and filters out functionally equivalent tests. All these adaptations can be included in AutoVRT, depending on the developer's preference. However, the chosen sequence and the particular actions for the OSTTRA React component library are effective.

In the AutoVRT implementation, an XPath locator is utilised during state exploration. The locator relies on XPath to identify which elements are interactive. This approach assumes that the code is written correctly and that only elements intended for interaction can be interacted with. For example, if the code uses a div with a click handler to create an interactive element, the state exploration may not be able to detect it. As a result, AutoVRT is most effective when the code follows code conventions. However, this limitation can also be viewed as a strength. With the growing emphasis on accessibility with the new EAA legislation [2], which requires components to be correctly implemented for an accessible web experience, this limitation can motivate developers to produce high-quality code.

6.2 Generalisability of Mutation Operators

We anticipated that traditional mutation operators are not suitable [33] for our testing process from the onset of our research. Further into the analysis, it becomes increasingly clear that mutating CSS provides a promising alternative approach. With Playwright's ability to detect a wide range of CSS properties, we shift our focus to identifying properties based on their frequency within the component library.

Through our examination, we determine that colour properties emerge as the most prevalent in our study subject, i.e., the React component library at OSTTRA, leading us to prioritise these for our mutation strategy. We could have used other common CSS properties to mutate, such as width and font-size. However, since these properties are less commonly used, they would result in less coverage in our mutation analysis, ultimately leading to a poorer evaluation of our fault-detecting capabilities. It is important to note that if we had been working with a different study subject, our selection of properties for mutation might have varied significantly. We can customise AutoVRT to target mutation operators that suit the specific library. The only constraint on the types of mutants our tool can target is that they must be expressible as regular expressions.

This adaptability is a key aspect of mutation testing, as it underscores the necessity of tailoring approaches to the specific characteristics of the codebase in question [26]. Regarding our approach, we perform mutations directly on the source code rather than on compiled outputs. This allows us to mutate one file per component rather than mutating the code of each Storybook story. However, this decision reinforces the importance of having access to the original source code, as the success of our mutation testing hinges on it. Thus, our approach requires direct interaction with the CSS code, highlighting a limitation: this mutation testing tool is best suited for component libraries where the CSS is readily available.

However, this limitation should not be viewed as a hindrance. Given that this tool is specifically designed for component libraries, it fits into developers' workflows, who are often the primary users interested in conducting thorough testing of their components. Developers are typically more engaged with aspects of the source code, including CSS, making this approach quite relevant.

In contrast, other studies in the domain of VRT often adopt a broader scope by analysing entire web pages rather than isolated components [9]. These approaches frequently rely on manipulating the Document Object Model (DOM) directly to inject mutants, bypassing the need for access to the source code [35]. While this flexibility can be advantageous in black-box settings, it introduces significant complexity in accurately crawling and manipulating the DOM to simulate realistic faults. Furthermore, testing at the page level inherently limits coverage to only those components that are rendered in specific views. As a result, components that exist in the codebase but are not actively used or visible in a given test scenario remain untested. In contrast, component-level testing, as in our study, allows for systematic and exhaustive evaluation of each UI element in isolation, enabling more targeted fault injection and improved test coverage across the entire component library.

6.3 Fault Detection Abilities of Visual Regression Test Suites

Upon reviewing the results for RQ4, we can see a general increase in the mutation score from the original VRT compared to the improved version. This suggests that the improved VRT suite has better fault-detecting capabilities than the original one, and thus, the generated tests make a notable difference. In the Venn diagram (see Figure 5.5), it is evident that the tests utilising interaction eliminated most of the detected mutants. However, the mutation scores, which are 0.22 before and 0.53 after (see Table 5.5), remain below 1.00 and thus cannot be considered mutation adequate (see section 2.5).

Several factors explain the results. It is clear that state exploration through interaction successfully uncovers previously untested states, leading to the generation of more impactful tests. However, not all mutants are detected, and there are specific reasons for this. Some of these reasons are related to the way state exploration is implemented, which will be examined in more detail in the following section. The remaining factors will be discussed below.

Consider the example with the Switch component impacted by a mutation mentioned in chapter 5.4. This mutant affects the appearance of the component when it is both checked and disabled (see Figure 5.7). This means that the Switch must either be checked while in a disabled state from the start or

become disabled after being checked. Although there is a Storybook story where the Switch is disabled, there is no version where it is both checked and disabled at the same time. Consequently, the original VRT is unable to detect this mutation. Furthermore, since it is impossible to interact with a disabled component, the generated tests do not identify this mutant either, as AutoVRT can only find states accessible through interaction.

This highlights a significant limitation of both the original and improved VRT. The original VRT relies solely on the initial state of each Storybook story, and since AutoVRT uses these stories as entry points for state exploration, both versions depend heavily on the number and selection of stories defined by the developers. Currently, AutoVRT cannot modify input parameters such as event handlers or whether a component is disabled. This presents an interesting avenue for further exploration. However, if such modifications are not implemented, we must consider the following questions: How many stories should be used for each component? Is it necessary to test all possible combinations of properties? Could a low mutation score indicate that there are too few stories of the component defined?

For example, the Dropdown component performs the worst both before and after the tests. This is likely due to its complexity, while it doesn't involve many intricate interactions, it supports extensive customisation. The component is a clickable box that displays a list of options when clicked. Despite this, the Storybook only presents a limited subset of its features, offering just four variants: condensed, disabled, staff, and standard. Many available options are not represented, such as different styles (e.g., primary, negative, attention), grouping of options, and size variations. As a result, only a small portion of the component's full functionality is covered, which explains why only six tests are generated. In contrast, the Switch component, while less customisable, achieves the same number of tests and shows the greatest improvement in mutation score. A possible reason is that the Switch stories cover a larger percentage of its available variations, leading to more comprehensive testing and better performance.

A general trend can be observed between the number of generated tests and the number of component stories: components with more stories tend to result in more generated tests. This is because each story represents a variation of a component, often with different properties, states, or visual configurations. The more variations available, the more distinct states can be explored through interactions. For example, consider a button component that supports multiple

interactions. Each interaction can reveal a different visual state. If multiple stories exist that present the button with different styles or configurations, even if they are functionally similar, they will still lead to additional tests being generated. This is because AutoVRT doesn't consider functional equivalence, and each story is treated as a unique component and therefore requires distinct tests.

However, no clear correlation can be observed between the number of generated tests and the magnitude of improvement in mutation score after enhancement, even though it is clear that the generated tests improve the mutation score. As discussed, this is likely more to do with how large the stories cover a percentage of the component's functionality. Furthermore, many of the generated tests may also be functionally equivalent, that is, they interact with the component in the same way, even if they originate from different stories. For instance, consider a component that gets a border when focused. Multiple stories can illustrate this component in various forms. These variations will have separate tests to verify this interaction, resulting in a border. Consequently, they might all detect the same mutant that affects the border, rather than increasing the total number of mutants detected. Thus, a higher number of tests does not necessarily translate to a proportionally higher mutation score improvement.

Although there is no straightforward correlation between the number of tests and the resulting mutation score, AutoVRT consistently improves mutation scores across all evaluated components. This highlights its potential for broader applicability across other React component libraries that integrate with Storybook. To the author's knowledge, there is currently no tool designed explicitly for component libraries that automatically generates tests suitable for both static and interactive components. Notably, AutoVRT does not require any pre-existing Playwright setup or manually written tests, as it automatically generates baseline tests during the execution of the Stories unit (see Section 3.1.1.1). If there are no pre-existing tests, the mutation analysis part will only be performed on the generated tests.

The effectiveness of AutoVRT in generating impactful tests depends largely on two factors: the comprehensiveness of the Storybook stories, how well they represent various states and configurations of components and the selection of user actions explored during testing.

The reliance on Storybook brings notable advantages. It enables component-

level isolation, avoiding the need for DOM fragmentation or heuristic-driven component extraction techniques commonly employed in other VRT studies [9, 36]. This isolation supports a more modular and scalable approach to visual regression. However, this reliance also introduces limitations: the testing depth is inherently tied to the quality and breadth of the available stories. If important states or configurations are omitted from the stories, they remain untested.

Moreover, the selection of mutation operators is central to evaluating the fault-detection capabilities of the VRT suite. Operators must be customised to the visual characteristics and functional behaviours of the component library. Prior research in mutation testing for UI and CSS-based systems underscores the importance of choosing representative mutants that mirror real-world visual faults [35, 26]. A poorly chosen mutation strategy risks underestimating the potential of the VRT suite.

To maximise AutoVRT's effectiveness, an analysis of the component library, similar to the one conducted in this study, is recommended. Such an analysis should focus on identifying relevant user interactions, determining the optimal sequence of actions, and selecting appropriate mutation operators tailored to the specific characteristics of the component library under evaluation.

Chapter 7

Conclusions and Future Work

In this chapter, we present the conclusions of the discussion and list possible directions for future work on the generation of visual regression tests.

7.1 Conclusions

This study investigates how automated state exploration through interaction can enhance the visual regression test suite for a component library. We develop AutoVRT, a two-phase tool that can be integrated with popular test frameworks Storybook and Playwright. During Phase I, AutoVRT utilises interaction during state exploration to identify untested states, while in Phase II, it evaluates the fault-finding effectiveness of the generated tests using mutation analysis.

We evaluate AutoVRT against an industrial study subject, a React component library developed at OSTTRA AB. To determine the actions that should be employed in the state exploration, we conduct an analysis of the component library. This analysis reveals that the most common actions are clicking, hovering, and focusing, which are subsequently utilised in the state explorations. In order to select appropriate mutants for this context, we perform a thorough analysis of both the source code of the component library and relevant literature. Through our investigation of potential mutants, we determine that modifying colour properties is the most appropriate approach

for our study subject.

Overall, our exploration and analysis of suitable mutations and actions to use in the state exploration emphasise the importance of understanding the specific environment in which a testing tool operates. This ensures that the deployment of AutoVRT aligns with the characteristics and needs of the analysed component libraries.

AutoVRT was applied to five components, resulting in the generation of 86 new interaction-based tests. The outcomes of the evaluation phase of AutoVRT showed that the total mutation scores improved from 0.22 for the original version to 0.53 for the improved version, indicating an average enhancement of 130% per component. However, the overall score still reflects some inadequacies in the mutation analysis. These observed limitations underscore the necessity for well-defined stories that thoroughly present the various states of the components. This is crucial for effective state exploration and test generation.

7.2 Limitations

In this study, we focus on a component library that exclusively contains React components. The library uses Storybook [21], an open-source tool for documenting and showcasing UI components, and a VRT suite that employs Playwright. Our current implementation and evaluation are thus limited to React component libraries that use both tools. The primary goal of this study is to serve as a proof of concept for enhancing VRT suites through the incorporation of interaction-based test generation. This approach enables us to create more dynamic and realistic testing scenarios that go beyond simple visual comparisons.

7.3 Future Work

This study serves as a proof of concept aimed at enhancing the fault-detecting capabilities of VRT suites for component libraries. It aims to address common challenges users face when interacting with various components by utilising

a limited sample of five components to illustrate this concept. However, there is significant potential for future research to build upon these findings by incorporating a broader range of components into the analysis.

Expanding the number of components will provide a more comprehensive understanding of how VRTs can be optimised for different use cases and design patterns within various libraries. Additionally, future investigations could further analyse the interaction dynamics among multiple layers of components and component with infinite loops, allowing researchers to experiment with diverse exploration strategies that could yield deeper insights into component behaviour.

One area for potential further exploration is to incorporate functionality that differentiates between functionally equivalent tests effectively, improving the accuracy and reliability of regression testing processes. This approach could reduce the number of tests, keeping only the most impactful and increase the maintainability of the test suite. Moreover, another interesting direction for future research could involve the implementation of visual locators as opposed to traditional CSS-based selectors. Using visual locators can enhance test robustness regardless of the underlying HTML structure.

Finally, it would be beneficial to extend this proof of concept to other component libraries. Testing this framework across a variety of libraries would help validate its effectiveness and adaptability, providing valuable insights into how VRTs can be universally applied to improve user experiences and mitigate common issues experienced with component interaction. It would be interesting to generalise the methodology for other component libraries where different actions and CSS properties are dominant. The findings from this expanded research could ultimately lead to the development of more versatile and resilient testing frameworks that cater to diverse requirements in software development.

References

[1] K. Ivanova, G. Kondratenko, I. Sidenko, and Y. Kondratenko, "Artificial Intelligence in Automated System for Web-Interfaces Visual Testing," *International Conference on Computational Linguistics and Intelligent Systems*, 2020. [Page 1.]

[2] "European accessibility act - European Commission." [Online]. Available: https://commission.europa.eu/strategy-and-policy/policies/justice-and-fundamental-rights/disability/union-equality-strategy-rights-persons-disabilities-2021-2030/european-accessibility-act_en [Pages 1 and 60.]

[3] F. Macchi, P. Rosin, J. M. Mervi, and L. Turchet, "Image-based approaches for automating GUI testing of interactive web-based applications," *Conference of Open Innovation Association, FRUCT*, vol. 2021-January, 1 2021. doi: 10.23919/FRUCT50888.2021.9347592. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9347592 [Pages 1 and 7.]

[4] F. Huang, "FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO Visual Regression Testing in Practice: Problems, Solutions, and Future Directions," Tech. Rep., 2024. [Pages 1, 9, and 17.]

[5] R. Yandrapally and A. Mesbah, "Mutation Analysis for Assessing End-to-End Web Tests," 2021. doi: 10.1109/ICSME52107.2021.00023 [Pages 2 and 15.]

[6] "Fast and reliable end-to-end testing for modern web apps | Playwright." [Online]. Available: https://playwright.dev/ [Pages 2, 8, 9, and 12.]

[7] "Visual testing & review for web user interfaces • Chromatic." [Online]. Available: https://www.chromatic.com/ [Pages 2 and 8.]

[8] "Percy | Visual testing as a service." [Online]. Available: https://percy.io/ [Pages 2 and 8.]

[9] R. Krishna Yandrapally, S. Member, and A. Mesbah, "Fragment-Based Test Generation for Web Apps," 2022. doi: 10.1109/TSE.2022.3171295. [Online]. Available: https://ieeexplore.ieee.org/document/9765776 [Pages 2, 3, 17, 19, 62, and 65.]

[10] S. Panda and D. P. Mohapatra, "Regression test suite minimization using integer linear programming model," *Software: Practice and Experience*, vol. 47, no. 11, pp. 1539–1560, 11 2017. doi: 10.1002/SPE.2485. [Online]. Available: /doi/pdf/10.1002/spe.2485https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.2485https://onlinelibrary.wiley.com/doi/10.1002/spe.2485 [Page 2.]

[11] Y. Adachi, H. Tanno, and Y. Yoshimura, "A Method to Mask Dynamic Content Areas Based on Positional Relationship of Screen Elements for Visual Regression Testing," *Proceedings - 2020 IEEE 44th Annual Computers, Software, and Applications Conference, COMPSAC 2020*, pp. 1755–1760, 7 2020. doi: 10.1109/COMPSAC48688.2020.000-1 [Pages 7 and 9.]

[12] "GitHub - garris/BackstopJS: Catch CSS curve balls." [Online]. Available: https://github.com/garris/BackstopJS [Page 8.]

[13] "GitHub - oblador/loki: Visual Regression Testing for Storybook." [Online]. Available: https://github.com/oblador/loki [Page 8.]

[14] T. Saar, M. Dumas, M. Kaljuve, and N. Semenenko, "Browserbite: Cross-Browser Testing via Image Processing," 3 2015. [Online]. Available: https://arxiv.org/abs/1503.03378v1 [Page 9.]

[15] "GitHub - mapbox/pixelmatch: The smallest, simplest and fastest JavaScript pixel-level image comparison library." [Online]. Available: https://github.com/mapbox/pixelmatch [Pages 9, 13, and 24.]

[16] "WebDriver | Selenium." [Online]. Available: https://www.selenium.dev/documentation/webdriver/ [Page 9.]

[17] "Testing Frameworks for Javascript | Write, Run, Debug | Cypress." [Online]. Available: https://www.cypress.io/ [Page 9.]

[18] "MUI: The React component library you always wanted." [Online]. Available: https://mui.com/ [Page 9.]

[19] "Chakra UI." [Online]. Available: https://chakra-ui.com/ [Page 9.]

[20] "Ant Design - The world's second most popular React UI framework." [Online]. Available: https://ant.design/ [Page 9.]

[21] "Storybook: Frontend workshop for UI development." [Online]. Available: https://storybook.js.org/ [Pages 10 and 68.]

[22] M. Praneeth Reddy, "Analysis of Component Libraries for React JS," *IARJSET*, vol. 8, no. 6, pp. 43–46, 6 2021. doi: 10.17148/IARJSET.2021.8607 [Page 10.]

[23] V. Anand and D. Saxena, "Comparative study of modern web browsers based on their performance and evolution," *2013 IEEE International Conference on Computational Intelligence and Computing Research, IEEE ICCIC 2013*, 2013. doi: 10.1109/ICCIC.2013.6724273 [Page 12.]

[24] V. N. Do, Q. V. Nguyen, and T. B. Nguyen, "Evaluating Mutation Operator and Test Case Effectiveness by Means of Mutation Testing," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 12672 LNAI, pp. 837–850, 2021. doi: 10.1007/978-3-030-73280-6_66/TABLES/5. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-030-73280-6_66 [Pages 15 and 16.]

[25] H. Du, V. K. Palepu, and J. A. Jones, "To Kill a Mutant: An Empirical Study of Mutation Testing Kills," *ISSTA 2023 - Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 715–726, 7 2023. doi: 10.1145/3597926.3598090. [Online]. Available: https://dl.acm.org/doi/pdf/10.1145/3597926.3598090 [Page 15.]

[26] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. L. Traon, and M. Harman, "Mutation Testing Advances: An Analysis and Survey," *Advances in Computers*, vol. 112, pp. 275–378, 1 2019. doi: 10.1016/BS.ADCOM.2018.03.015. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0065245818300305 [Pages 15, 18, 61, and 65.]

[27] A. J. Offutt, A. Lee, G. Rothermel, R. H. Untch, and C. Zapf, "An Experimental Determination of Sufficient Mutant Operators," *ACM Transactions on Software Engineering and Methodology*, vol. 5, no. 2, pp. 99–118, 4 1996. doi: 10.1145/227607.227610/ASSET/201DF323-9D28-40DD-9C75-3E26E7FDC56B/ASSETS/227607.227610.FP.PNG. [Online]. Available: https://dl.acm.org/doi/10.1145/227607.227610 [Page 16.]

[28] F. Ricca, M. Leotta, and A. Stocco, "Three Open Problems in the Context of E2E Web Testing and a Vision: NEONATE," *Advances in Computers*, vol. 113, pp. 89–133, 1 2019. doi: 10.1016/BS.ADCOM.2018.10.005. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0065245818300652#s0170 [Page 17.]

[29] S. Thummalapenta, K. V. Lakshmi, S. Sinha, N. Sinha, and S. Chandra, "Guided test generation for web applications," *Proceedings - International Conference on Software Engineering*, pp. 162–171, 2013. doi: 10.1109/ICSE.2013.6606562 [Page 18.]

[30] J. W. Lin, N. Salehnamadi, and S. Malek, "Route: Roads Not Taken in UI Testing," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 3, 4 2023. doi: 10.1145/3571851/ASSET/10F33D6D-BF20-4284-8187-520FABFA55C3/ASSETS/GRAPHIC/TOSEM-2021-0307-F12.JPG. [Online]. Available: https://dl.acm.org/doi/10.1145/3571851 [Page 18.]

[31] M. Leotta, A. Stocco, F. Ricca, and P. Tonella, "Pesto: Automated migration of DOM-based Web tests towards the visual approach," *Software Testing Verification and Reliability*, vol. 28, no. 4, p. e1665, 6 2018. doi: 10.1002/stvr.1665 [Page 18.]

[32] S. Balsam and D. Mishra, "Web application testing—Challenges and opportunities," *Journal of Systems and Software*, vol. 219, p. 112186, 1 2025. doi: 10.1016/J.JSS.2024.112186 [Page 18.]

[33] R. A. Oliveira, E. Alégroth, Z. Gao, and A. Memon, "Definition and evaluation of mutation operators for GUI-level mutation analysis," *2015 IEEE 8th International Conference on Software Testing, Verification and Validation Workshops, ICSTW 2015 - Proceedings*, 5 2015. doi: 10.1109/ICSTW.2015.7107457 [Pages 18, 39, 43, and 61.]

[34] L. Deng, J. Offutt, P. Ammann, and N. Mirzaei, "Mutation operators for testing Android apps," *Information and Software Technology*, vol. 81, pp. 154–168, 1 2017. doi: 10.1016/J.INFSOF.2016.04.012 [Pages 19 and 43.]

[35] M. Tafreshipour, A. Deshpande, F. Mehralian, I. Ahmed, and S. Malek, "Ma11y: A Mutation Framework for Web Accessibility Testing," *ISSTA 2024 - Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 100–111, 9 2024. doi: 10.1145/3650212.3652113. [Online]. Available: https://dl.acm.org/doi/10.1145/3650212.3652113 [Pages 19, 39, 43, 62, and 65.]

[36] R. Yandrapally, S. Sinha, R. Tzoref-Brill, and A. Mesbah, "Carving UI Tests to Generate API Tests and API Specification," *Proceedings - International Conference on Software Engineering*, pp. 1971–1982, 2023. doi: 10.1109/ICSE48619.2023.00167 [Page 65.]

TRITA-EECS-EX-2025:405
Stockholm, Sweden 2025